
O-RAN Work Group 2 (Non-Real-time RIC and A1 Interface)

Study on A1 policy conflict mitigation

Copyright © 2025 by the O-RAN ALLIANCE e.V.

The copying or incorporation into any other work of part or all of the material available in this document in any form without the prior written permission of O-RAN ALLIANCE e.V. is prohibited, save that you may print or download extracts of the material of this document for your personal use, or copy the material of this document for the purpose of sending to individual third parties for their information provided that you acknowledge O-RAN ALLIANCE as the source of the material and that you inform the third party that these conditions apply to them and that they must comply with them.

O-RAN ALLIANCE e.V., Buschkauler Weg 27, 53347 Alfter, Germany
Register of Associations, Bonn VR 11238, VAT ID DE321720189

Contents

Foreword	4
Modal verbs terminology	4
1 Scope	5
2 References	5
2.1 Normative references	5
2.2 Informative references	5
3 Definition of terms, symbols and abbreviations	5
3.1 Terms	5
3.2 Symbols	5
3.3 Abbreviations	6
4 Overview	6
4.1 Background	6
4.2 Concept	6
4.2.1 A1 policy conflict	6
4.2.1.1 Definition	6
4.2.1.2 General description	7
5 Use cases	7
5.1 Use case #1 “reserve policy” vs. “release policy”	7
5.1.1 Description	7
5.1.2 Some representative scenarios	8
5.2 Use case #2 Policy objectives not uniquely identified.	9
5.2.1 Description	9
5.2.2 Some representative scenarios	9
5.3 Use case #3 Policy resources preference not uniquely identified.	10
5.3.1 Description	10
5.3.2 Some representative scenarios	10
6 Detection of A1 policy conflict	10
6.1 General	10
6.2 Detection of overlaps with policy scope and policy resource	11
6.2.1 A1 policy applicable entity type	12
6.2.2 Example criteria for finding overlaps with policy scope or policy resource.	17
6.2.3 Examples of overlap in policy scope	19
6.2.4 Overlap information of A1 policy scopes and policy resources	25
6.3 Detection of conflicting situations	25
6.3.1 Evaluation of conflicting situation for collection of overlaps	25
6.3.2 Approach to evaluate conflicting situation	26
6.3.3 Information of conflicting situations	28
6.4 A1 policy conflict detection service	28
7 A1 policy conflict avoidance	32
7.1 General	32
7.2 A1 policy conflict arbitration service	33
7.2.1 A1 policy conflict arbitration - Option #1. Handle in sequence	34
7.2.1.1 Procedures	34
7.2.2 A1 policy conflict arbitration - Option #2. Priority, Preemption protection	35
7.2.2.1 Functionality analysis	35
7.2.2.2 Procedures	37
7.3 Procedures	38
7.3.1 General	38
7.3.2 Approach #1. enforce A1 policy based on arbitration result	40
7.3.3 Approach #2. multiple policies co-enforced with priorities and pre-emption information	44
7.3.4 Approach #3. provide alternative conflict-free policies	44

7.3.5	Analysis of approaches	45
8	Recommendations	46
8.1	Recommendations for A1 policy conflict related services	46
8.2	Recommendations for A1 policy management services	47
8.3	Note for recommendations in terms of formulation for A1 policy conflict related services	48
	Change history / Change request (history)	49

Foreword

This Technical Report (TR) has been produced by O-RAN Alliance.

The content of the present document is subject to continuing work within O-RAN and may change following formal O-RAN approval. Should the O-RAN Alliance modify the contents of the present document, it will be re-released by O-RAN with an identifying change of version date and an increase in version number as follows:

version xx.yy.zz

where:

- xx: the first digit-group is incremented for all changes of substance, i.e. technical enhancements, corrections, updates, etc. (the initial approved document will have xx=01). Always 2 digits with leading zero if needed.
- yy: the second digit-group is incremented when editorial only changes have been incorporated in the document. Always 2 digits with leading zero if needed.
- zz: the third digit-group included only in working versions of the document indicating incremental changes during the editing process. External versions never include the third digit-group. Always 2 digits with leading zero if needed.

Modal verbs terminology

In the present document "**shall**", "**shall not**", "**should**", "**should not**", "**may**", "**need not**", "**will**", "**will not**", "**can**" and "**cannot**" are to be interpreted as described in clause 3.2 of the O-RAN Drafting Rules (Verbal forms for the expression of provisions).

"**must**" and "**must not**" are **NOT** allowed in O-RAN deliverables except when used in direct citation.

1 Scope

The present document contains considerations and recommendations related to A1 policy conflict mitigation.

2 References

2.1 Normative references

Not applicable.

2.2 Informative references

References are either specific (identified by date of publication and/or edition number or version number) or non-specific. For specific references, only the cited version applies. For non-specific references, the latest version of the referenced document (including any amendments) applies.

NOTE: While any hyperlinks included in this clause were valid at the time of publication, O-RAN cannot guarantee their long term validity.

The following referenced documents are not necessary for the application of the present document but they assist the user with regard to a particular subject area.

- [i.1] O-RAN TS: "A1 interface: General Aspects and Principles" ("A1GAP").
- [i.2] O-RAN TS: "Non-RT RIC: Architecture" ("Non-RT-RIC-ARCH").
- [i.3] O-RAN TS: "R1 interface: General Aspects and Principles" ("R1GAP").
- [i.4] O-RAN TS: "A1 interface: Type Definitions" ("A1TD").
- [i.5] O-RAN TS: "E2 Service Model" ("E2SM").

3 Definition of terms, symbols and abbreviations

3.1 Terms

For the purposes of the present document, the following terms apply:

E2SM: E2 Service Model

Near-RT RIC: Near-Real Time RIC

Non-RT RIC: Non-Real Time RIC

3.2 Symbols

Void

3.3 Abbreviations

For the purposes of the present document, the following abbreviations apply:

PAET: A1 policy applicable entity type

4 Overview

4.1 Background

Within O-RAN, A1 Policies and E2SMs [i.5] IE/Objects enable the intelligent guidance of the RAN towards better fulfilment of the RAN Intent. Non-RT RIC and Near-RT RIC, each has its own context that need to be broken down into more detailed ones and to be passed to next lower layer as shown in Figure 4.1-1.

NOTE: The RAN intent is outside the scope of the A1 interface.

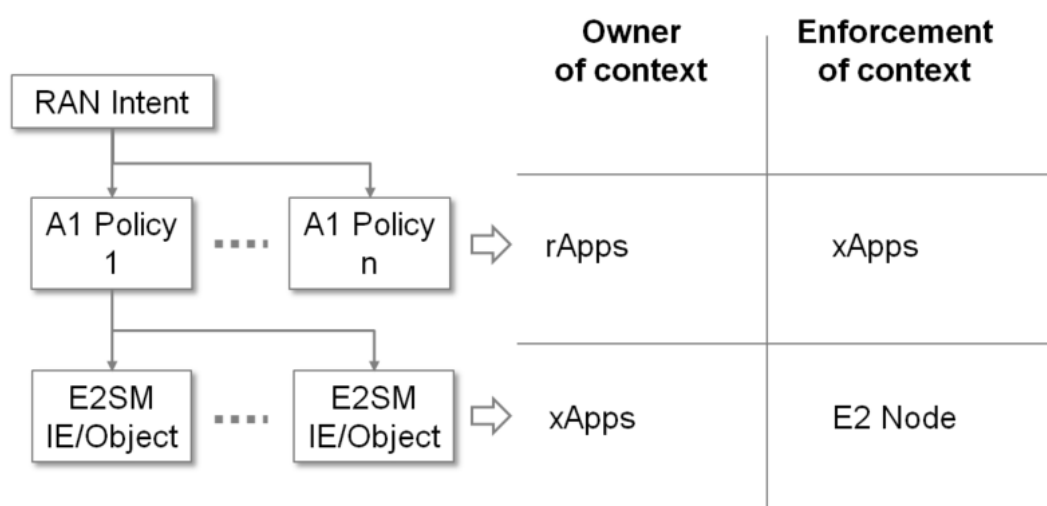


Figure 4.1- 1: Context for each layer

The purpose of the A1 policies is to guide the RAN towards the overall goal expressed in RAN intent in A1GAP [i.1]. As the service consumer of A1 policy management services, rApps can send A1 policies based on their own business purpose, and A1 policy conflict may happen there. Non-RT-RIC-ARCH [i.2] has defined conflict mitigation for A1 policies as follows: If more than one policy generation rApps set contradicting A1 policy statements to the same or overlapping policy scope identifiers, A1 policy function decides to reject the creation/modification request of one or more A1 policies or to temporarily suspend the enforcement of one or more A1 policies.

4.2 Concept

4.2.1 A1 policy conflict

4.2.1.1 Definition

A1 policy conflict: A situation where it is not feasible for a Near-RT RIC function, and hence the RAN to apply the guidance provided by a set of two or more A1 policies to concurrently achieve the policy objectives and/or to satisfy the conditions for policy resource usage.

4.2.1.2 General description

A1 policy conflict would be generally recognized in two ways; “actual conflict” and “potential conflict”. The actual A1 policy conflict would be detected and handled for created A1 policies at a Near-RT RIC while the potential A1 policy conflict would be predicted and mitigated for requested A1 policies over R1 interface at the Non-RT RIC before a new A1 policy is requested to be created or an existing policy at a Near-RT RIC is requested to be updated over A1 interface.

The potential A1 policy conflict would be predicted by analysing contents of a requested policy and existing policies to be concurrently enforced at a Near-RT RIC, and mitigated to avoid the policy conflict with possible approaches of resolutions.

This TR will study and describe ways to predict and mitigate potential A1 policy conflicts in Non-RT RIC, and this involves further study of A1 policy conflict mitigation functionality of A1 policy functions as specified in Non-RT-RIC-ARCH [i.2], clause 8.4.1.

NOTE: The potential A1 policy conflict is referred to as the A1 policy conflict or just the policy conflict, and the usage of term “predict” for policy conflict does not preclude the usage of term “detect” for policy conflict if not necessary to distinguish between them in the Technical Report.

5 Use cases

5.1 Use case #1 “reserve policy” vs. “release policy”

5.1.1 Description

Policies of different A1 policy types currently defined in A1TD [i.4], clause 7.2 could be conflicting with policy objectives and policy resources. Policies of different types with policy objectives (possibly with policy resources) such as QoS/QoE target, QoS/QoE optimization with resource directive, UE level target, Slice SLA target, Load balancing, Energy savings could be conflicting to achieve their objectives concurrently. Policies of the TSP type with policy resources such as Traffic steering preferences could be conflicting to satisfy the conditions for resource usage for the policy concurrently.

It is assumed that one of ways to categorize policy types is into mutually exclusive groups in terms of effect on RAN resource: “reserve policy” group and “release policy” group. The “reserve policy” group is for guaranteeing the service quality possibly by reserving resources, and the “release policy” group is for offloading cells and hence releasing the reserved resources. A general grouping of A1 policy types with clause numbers of A1TD [i.4], clause 7 into these groups is illustrated in Figure 5.1.1-1.

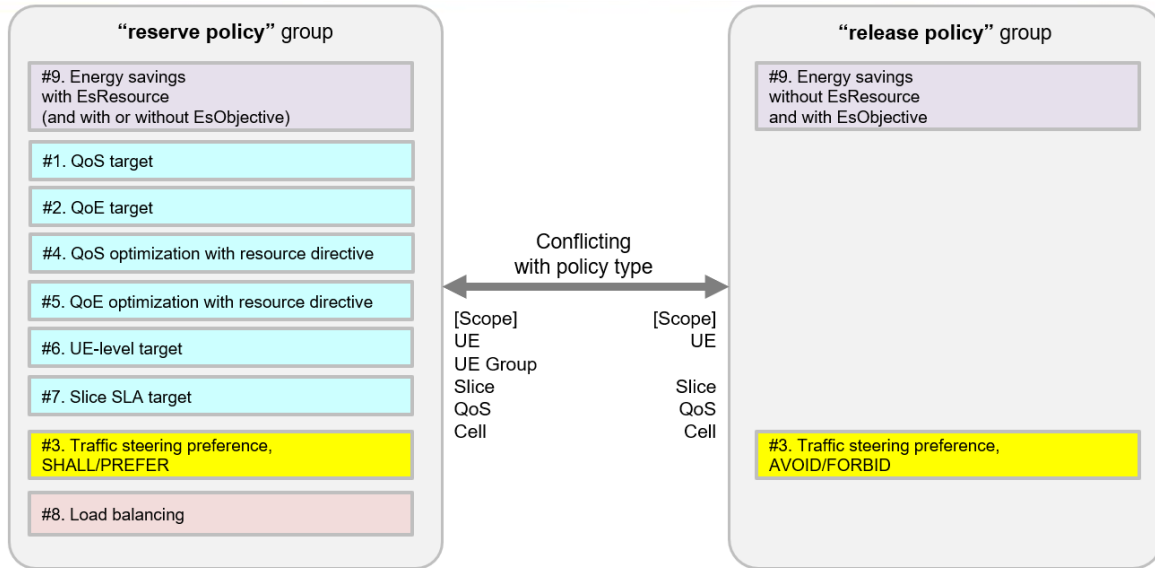


Figure 5.1.1- 1: Grouping of A1 policy types into “reserve policy” group and “release policy” group.

NOTE 1: A Traffic steering preferences policy is allowed to have both a cell list of SHALL/PREFER preference and a cell list of AVOID/FORBID preference. In this case the policy would have both characteristics of reserve policy and release policy groups with each specified scope.

NOTE 2: A Load balancing policy would have characteristic of reserve policy with traffic increase effect on specified cells and of release policy with traffic decrease effect on other specified cells in the middle of fulfilling the policy. At the stable condition brought by the LB policy there remains characteristic of reserve policy for both specified cells.

NOTE 3: An Energy savings policy would imply the coordination with appropriate traffic steering not to impair the user experience, and a policy without esObjectives would not be conflicting with reserve policy group resulting in best effort behaviour for energy savings. With esResource statement along with AVOID/FORBID, the possible energy saving schemes would be more strictly limited for specified cells, but it leaves to the Near-RT RIC whether it can perform the energy saving scheme (e.g. ASM) so as not to give impact on traffic or cell coverage as described in AITD [i.4] clause 6.3.4.5.

Co-existence of these different groups of A1 policies in a Near-RT RIC would have a conflicting situation for achieving their objectives.

5.1.2 Some representative scenarios

1. Service-guaranteed vs. Network energy saving without EsResource.

Service-guaranteed policies (QoS/QoE/UE-level/Slice_SLAs) guide to reserve RAN resources while NES without EsResource policies to release the same reserved RAN resources to expand its potential power-off duration.

2. Load balancing vs. Traffic steering preference with AVOID/FORBID

LB policies guide to reserve RAN resources at the target cells listed in the LbResources for distributing traffic load while TS preference policies to release the same reserved RAN resources at the target cells listed in the TspResources with AVOID/FORBID.

3. Traffic steering preference with SHALL/PREFER vs. Network energy saving without EsResource.

TS preference policies guide to reserve RAN resources at the target cells listed in the TspResources with SHALL/PREFER while NES without EsResource policies to release the same reserved RAN resources.

5.2 Use case #2 Policy objectives not uniquely identified.

5.2.1 Description

Entities in subset/intersection set relation of two or more different policies scopes in a Near-RT RIC would have a conflicting situation for identifying an unique policy objectives.

5.2.2 Some representative scenarios

1. UE-level vs. Slice SLA policies

In case that UE-level policy (ex. UL/DL RAN UE Throughput) and Slice SLA policy (ex. Max UL/DL Throughput per UE) specify the same entity in their scopes, it's not clear which of policies to be applied for that entity as illustrated in Figure 5.2.1-1.

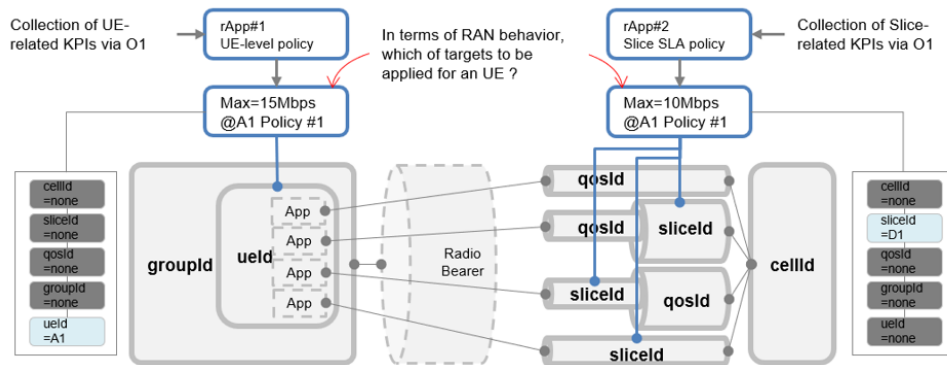


Figure 5.2.2- 1: Policy objectives not uniquely identified use case with UE-level vs. Slice SLA policies.

2. QoS target vs. Slice SLA policies

In case that QoS target policy (ex. UL/DL Guaranteed Flow Bit Rate) specifies a QoS flow in its scope that is impacted by a Slice SLA policy (ex. Max UL/DL Throughput per UE), it's not clear which of policies to be applied for the QoS flow as illustrated in Figure 5.2.1-2.

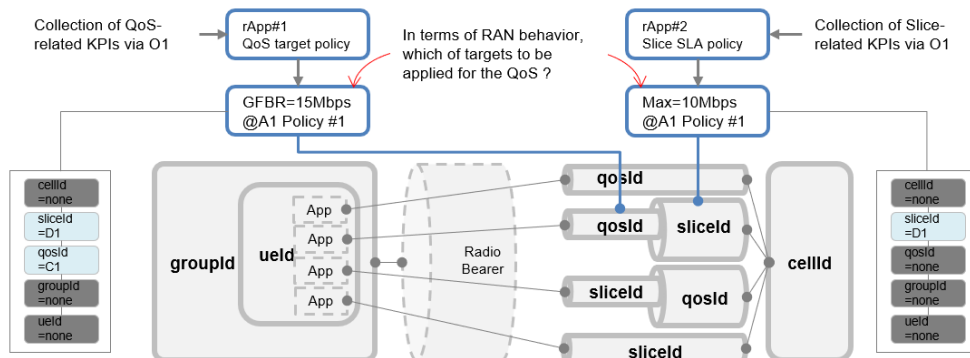


Figure 5.2.2- 2: Policy objectives not uniquely identified use case with QoS target vs. Slice SLA policies.

5.3 Use case #3 Policy resources preference not uniquely identified.

5.3.1 Description

Especially for the Traffic steering policy, UEs which are commonly specified with different “SHALL” preferences by different Traffic steering policies would have a conflicting situation for identifying a unique policy resources preference.

5.3.2 Some representative scenarios

1. TSP policy with ueId specified vs. TSP policy with qosId specified for different “SHALL” preference cells.

In case that there are two different TS policies; one for specified UE with {SHALL}={Cell-B} for optimizing UE performance and the other for specified QoS with {SHALL}={Cell-C} for optimizing QoS performance, UEs specified by TSP policy with ueId would have a conflicting situation for identifying a policy resources preference as illustrated in Figure 5.3.2-1 if conditions shown below are met.

- There can only be a conflict in case UEs for which policy#1 is applied has a session established for the qosId covered by policy#2.
- There can only be a conflict when the UEs are connected to cells that have a neighbourhood relation to the cells in cellList of policy 1 (cell-B) and the cells in cellList of policy#2 (cell-C).
- There can only be a conflict in case the cellLists with SHALL preference are disjoint.

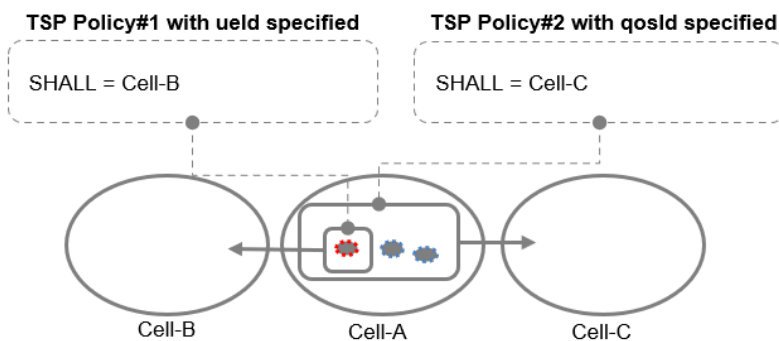


Figure 5.3.2- 1: TSP policy with ueId specified vs. TSP policy with qosId specified for different “SHALL” preference cells.

6 Detection of A1 policy conflict

6.1 General

The detection of A1 policy conflict would be performed when a new policy is requested to be created against a set of existing policies, or an existing policy is requested to be updated within a set of existing policies. The decision to regard a set of policies as a set of conflicting policies would be performed based on the following aspects:

1. to detect if there are overlapping scopes with their policy scopes or overlapping resources in their policy resources, and
2. to detect if A1 policies in each overlapping scope have potential conflict.

An example flowchart of A1 policy conflict detection is shown in Figure 6.1-1.

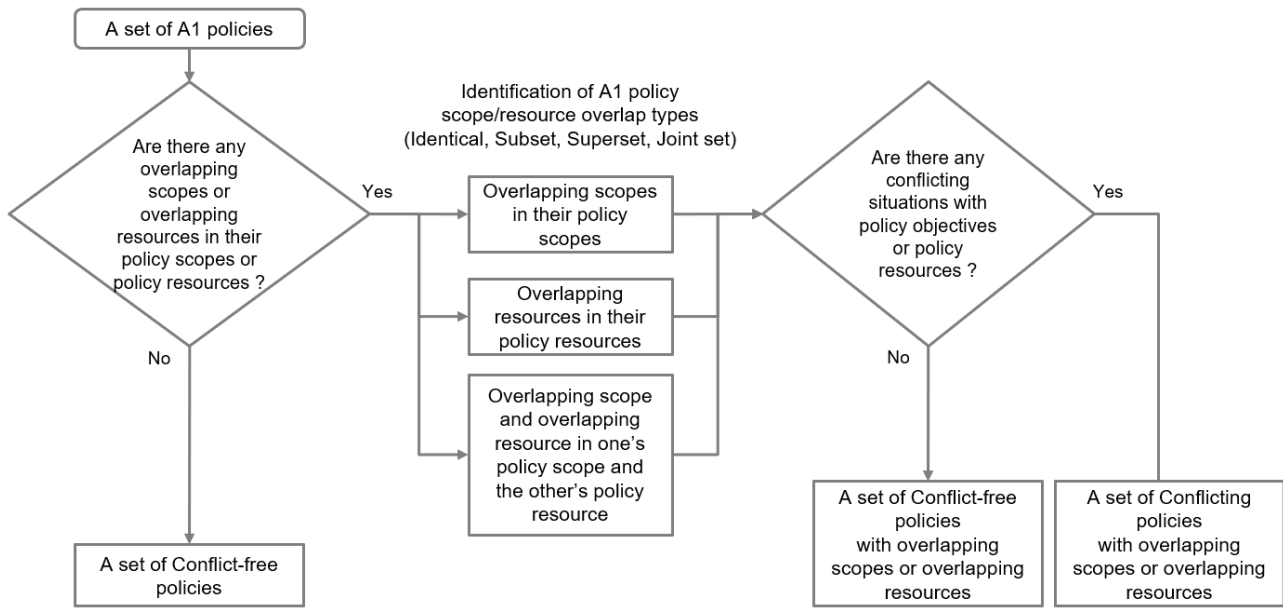


Figure 6.1- 1: An example flowchart of A1 policy conflict detection.

6.2 Detection of overlaps with policy scope and policy resource

Some examples of the overlapping scope with two policy scopes are illustrated in Figure 4.2.2.1-2. An example on top of the referred figure is a case of subset/superset relation with their policy scopes where scope of policy#1 is subset of scope of policy#2 or scope of policy#2 is superset of scope of policy#1. An example on bottom of the referred figure is a case of joint sets relation with their policy scopes where both scopes of policy#1 and policy#2 are joint sets having an intersection.

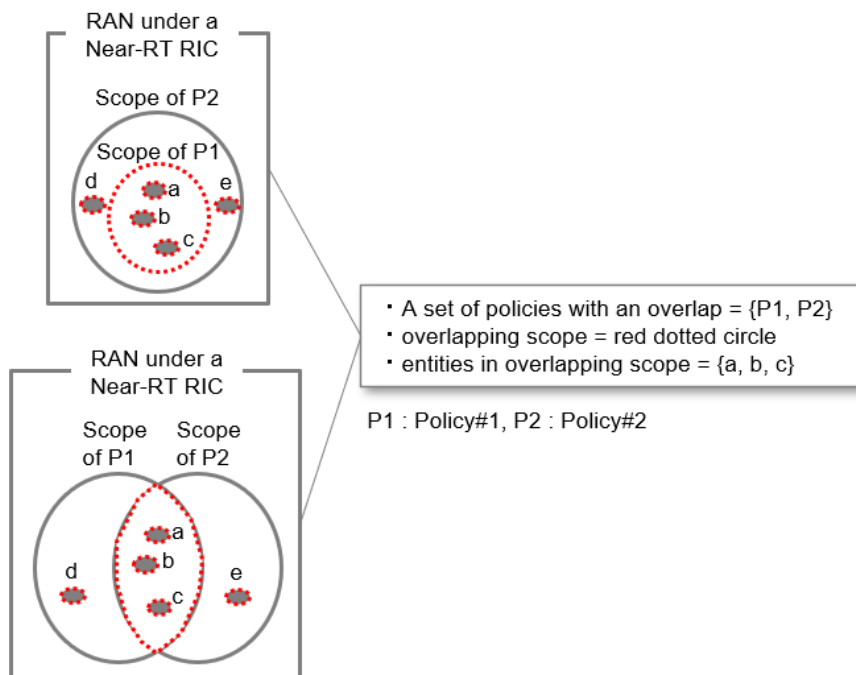


Figure 6.2- 1: Examples of the overlapping scope with two policy scopes

6.2.1 A1 policy applicable entity type

The ScopeIdentifier in an A1 policy contains the attributes (ueId, groupId, sliceId, qosId, cellId, cellIdList and taIList) defined in A1TD [i.4], clause 6.3.1.1, and also the policy statement contains the policy resources (TspResources, SliceSlaResources, LbResources, EsResource) defined in A1TD [i.4], clause 6.3.4 which include identification of policy resources with the attributes commonly used by the ScopeIdentifier (e.g., cellIdList in TspResource, cellIdList/taIList in SliceSlaResources, cellIdList in LbResources, cellIdList in EsResource).

Applicable entity is an entity which is derived explicitly or implicitly by policy scope, relevant granularity level of O1 observables corresponding to the policy objectives, and possibly by policy resources in a policy for identifying overlapping scope or overlapping resource between policies. An A1 policy would have at least one applicable entity type by policy scope and another applicable entity type possibly by policy resource. Based on the A1TD [i.4], clause 7.2, there are several types of A1 policy applicable entity to be derived from allowed combinations of attributes in ScopeIdentifier for policy types: explicit UE, implicit UE(s), explicit Slice and explicit Cell as shown in Table 4.2.2.2-1. As for A1 policy applicable entity derived by policy resource, there is only one type of explicit Cell.

Table 6.2.1- 1: Type of A1 policy applicable entity

Policy type	ueld	groupid	sliceld	qosld	cellld	cellldList	taiList	*1	*2
QoS target	1	0..1	0	1	0..1	-	-	explicit UE	Per UE
	1	0	0..1	1	0..1	-	-	explicit UE	
	0	1	0	1	0..1	-	-	implicit UE(s)	
	0	0	1	1	0..1	-	-	implicit UE(s)	
	0	0	0	1	0..1	-	-	implicit UE(s)	
QoE target	1	0	1	0..1	0..1	-	-	explicit UE	Per UE
	1	0	0	1	0..1	-	-	explicit UE	
	0	0	1	0..1	0..1	-	-	implicit UE(s)	
	0	0	0	1	0..1	-	-	implicit UE(s)	
TSP	1	0	0..1	0..1	0..1	-	-	explicit UE	Per Cell
	0	0	1	0..1	0..1	-	-	implicit UE(s)	
QoS target with TSP	1	0	0..1	1	0..1	-	-	explicit UE	Per UE
	0	0	1	0..1	0..1	-	-	implicit UE(s)	
QoE target with TSP	1	0	1	0..1	0..1	-	-	explicit UE	Per UE
	1	0	0	1	0..1	-	-	explicit UE	
	0	0	1	0..1	0..1	-	-	implicit UE(s)	
UE level target	1	0..1	0..1	0	0..1	-	-	explicit UE	Per UE
	1	0..1	0..1	1	0..1	-	-	explicit UE	
	1	0	1	0..1	0..1	-	-	explicit UE	
Slice SLA target	0	0	1	0	0	-	-	explicit Slice, implicit UE(s) for "per UE" policy targets	Per Slice
Load balancing	0	0	0	0	1	-	-	explicit Cell	Per Cell
	0	0	1	0	1	-	-	explicit Slice	Per Slice
Energy Savings	0	0	0	0	1	-	-	explicit Cell	Per Cell
	0	0	0	0	0	1	0	explicit Cell(s)	
	0	0	0	0	0	0	1	explicit Cell(s)	

(*1) Type of A1 policy applicable entity.

(*2) Type of O1 observables.

NOTE 1: Explicit UE is an entity type for which ueld is specified and optionally other attributes of the scopeldentifier would be specified.

NOTE 2: Implicit UE(s) is an entity type for which ueld is not specified and at least one or more other attributes of the scopeldentifier need to be specified and the relevant granularity level of O1 observables

corresponding to the policy objectives is per UE.

NOTE 3: Explicit Slice is an entity type for which sliceId is specified and both of uelId and groupId are not specified and the relevant granularity level of O1 observables corresponding to the policy objectives is per slice.

NOTE 4: Explicit Cell is an entity type for which only cellId or cellIdList or taiList of the scopeIdentifier is specified and the relevant granularity level of O1 observables corresponding to the policy objectives is per cell.

NOTE 5: Most of A1 policy applicable entities would have the corresponding O1 observables to be collected and evaluated by rApps.

NOTE 6: As for Traffic steering preference policy type, the policy preference would be per-cell and its corresponding per-cell O1 observables are collected and evaluated but the A1 policy applicable entities for TSP are assumed to be explicit UE or implicit UE(s) because actual affected controls such as handover would be applied to UEs.

NOTE 7: Tai element(s) in TaiList would be regarded as explicit cell because we assume Tai could be resolved into cellId(s) by R1 service and such resolution can be used in A1 policy management function.

NOTE 8: Implicit UE(s) in the Slice SLA target policy are a member of that slice which is connected to all possible cells under the Near-RT RIC.

ScopeIdentifiers of these entity types are illustrated in Figure 6.2.1-1.

ScopeIdentifier of Explicit UE (with per UE O1 observables)							
#	uelId	groupId	sliceId	qosId	cellId	cellIdList	taiList
1	1	Optionally specified					

ScopeIdentifier of Explicit Slice (with per Slice O1 observables)							
#	uelId	groupId	sliceId	qosId	cellId	cellIdList	taiList
1	0	0	1	0	Optionally specified		

ScopeIdentifier of Implicit UE(s) (with per UE O1 observables)							
#	uelId	groupId	sliceId	qosId	cellId	cellIdList	taiList
1	0	At least one or more other attributes specified					

ScopeIdentifier of Explicit Cell(s) (with per Cell o1 observables)							
#	uelId	groupId	sliceId	qosId	cellId	cellIdList	taiList
1	0	0	0	0	1	0	0
2	0	0	0	0	0	1	0
3	0	0	0	0	0	0	1

0... not specified,
1... specified

Figure 6.2.1- 1: ScopeIdentifier of A1 policy applicable entity type

These A1 policy applicable entity types (PAETs) would be used for finding an overlap in both policy scopes and policy resources and identify the types of overlaps. Set relations of policy scopes and/or policy resources with an overlap are shown below.

- **Identical** : The sets of applicable entities of both A1 policies are identical.
- **Subset** : A set of applicable entities for one non-identical A1 policy is a subset of applicable entities of the other A1 policy.
- **Superset** : A set of applicable entities for one non-identical A1 policy is a superset of applicable entities of the other A1 policy.
- **Joint sets** : There is a non-empty intersection between sets of applicable entities of two non-identical A1 policies.

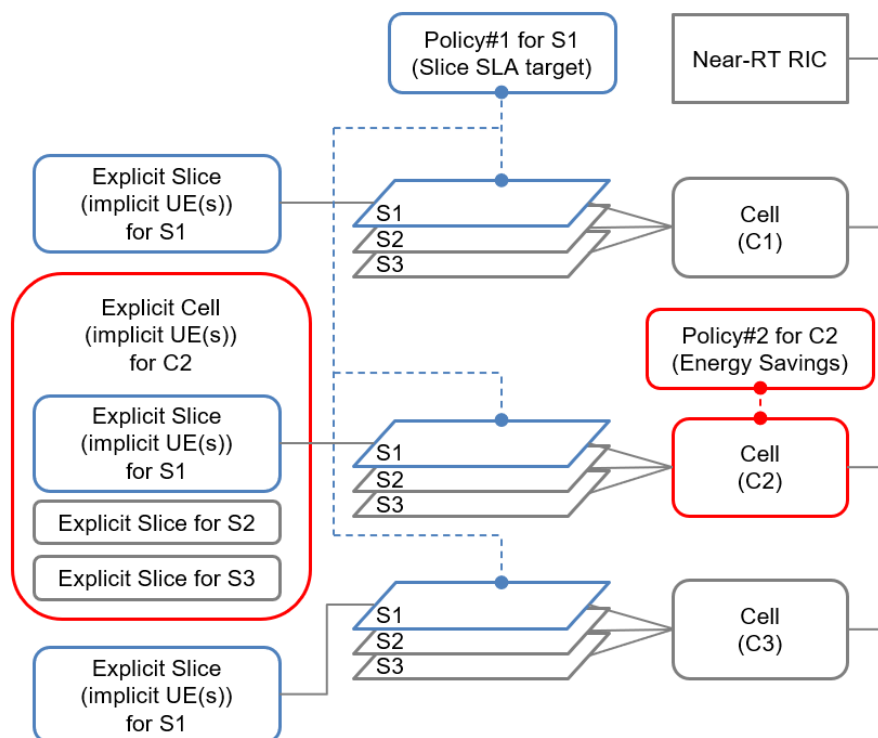
And possible set relations of PAETs with an overlap for a specific cell are shown in Table 6.2.1-2.

Table 6.2.1- 2: Possible set relations of PAETs with an overlap for a specific cell

	P1 is (xxx) with P2.	PAET#2 (P2)					
		explicit UE	Slice	Cell	implicit UE(s)		
PAET#1 (P1)	explicit	UE	ID	ID / SB	ID / SB	ID / SB	ID : Identical
		Slice	ID / SP	ID	ID / SB	ID / SP / JS	SB : Subset
		Cell	ID / SP	ID / SP	ID	ID / SP	SP : Superset
	implicit	UE(s)	ID / SP	ID / SB / JS	ID / SB	ID / SB / SP / JS	JS : Joint set

NOTE: Considering a case that single instance for each of UE, Slice and Cell, there would possibly be the case of identical (ID) relation of PAETs with an overlap for a specific cell even though two PAETs are different types.

In Figure 6.2.1-2, an example of the set relation of PAET#1 with explicit Slice for Policy#1 and PAET#2 with explicit Cell for Policy#2 with an overlap is shown. In this example, three cells (C1, C2 and C3) are controlled by the Near-RT RIC, three different slices (S1, S2 and S3) per cell are configured, and two policies (one for Slice SLA policy for slice S1 and the other for Energy Savings for cell C2) are applied. As referenced from Table 6.2.1-2, within the explicit cell C2, the set of entities for PAET#1 is subset of PAET#2, and its overlap is equal to the set of entities for PAET#1.


Figure 6.2.1- 2: An example of a PAET pair with explicit Slice and explicit Cell

In terms of relation with Slice and QoS, two typical cases of slice configurations are shown in Figure 6.2.1-3. One case is the application-based slice in which a slice instance is created per vertical application and multiple QoS flows related to that application are mapped into a slice. And the other case is the QoS-based slice in which a slice instance is created per a group of similar QoS characteristics. It should be noted that traffic flow with the same QoS flow ID is treated differently in the Application-based slice (e.g., App1's voice traffic with QoS flow ID Q1 is mapped into the slice S1 while App2's voice traffic with QoS flow ID Q1 is mapped into the slice S2.), and it is not in the QoS-based slice. In

Application-based slice

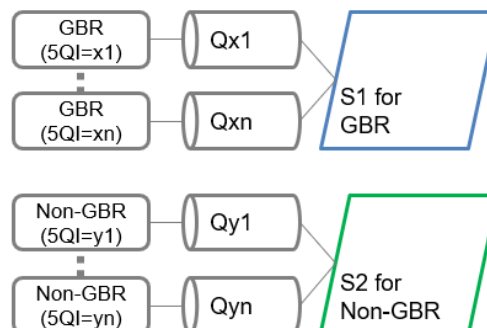
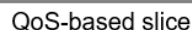
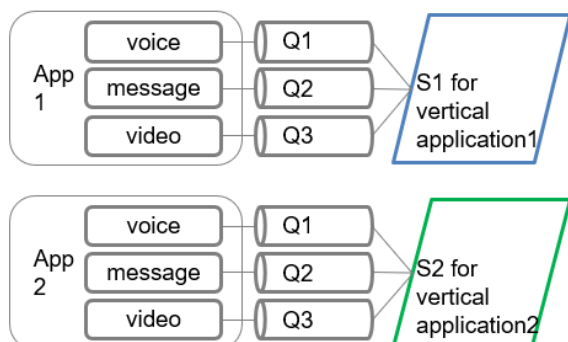


Figure 6.2.1- 3: Two typical cases of slice configurations

In Figure 6.2.1-4, an example of the set relation of PAET#1 with explicit Slice for Policy#1 and PAET#2 with implicit UE(s) for Policy#2 with an overlap is shown. In this example, three cells (C1, C2 and C3) are controlled by the Near-RT RIC, two different application-based slices (S1 and S2) per cell are configured, three QoS flows (Q1, Q2 and Q3) are handled in slices as shown in left side of Figure 6.2.1-3, and two policies (one for Slice SLA policy for Slice1 (S1) and the other for QoS target for QoS 1 (Q1)) are applied. As referenced from Table 6.2.1-2, within each of the explicit cells (C1, C2 and C3), there is an intersection of entities between the set of entities for PAET#1 and the set of entities for PAET#2, and its overlap is equal to the subset of entities for PAET#2.

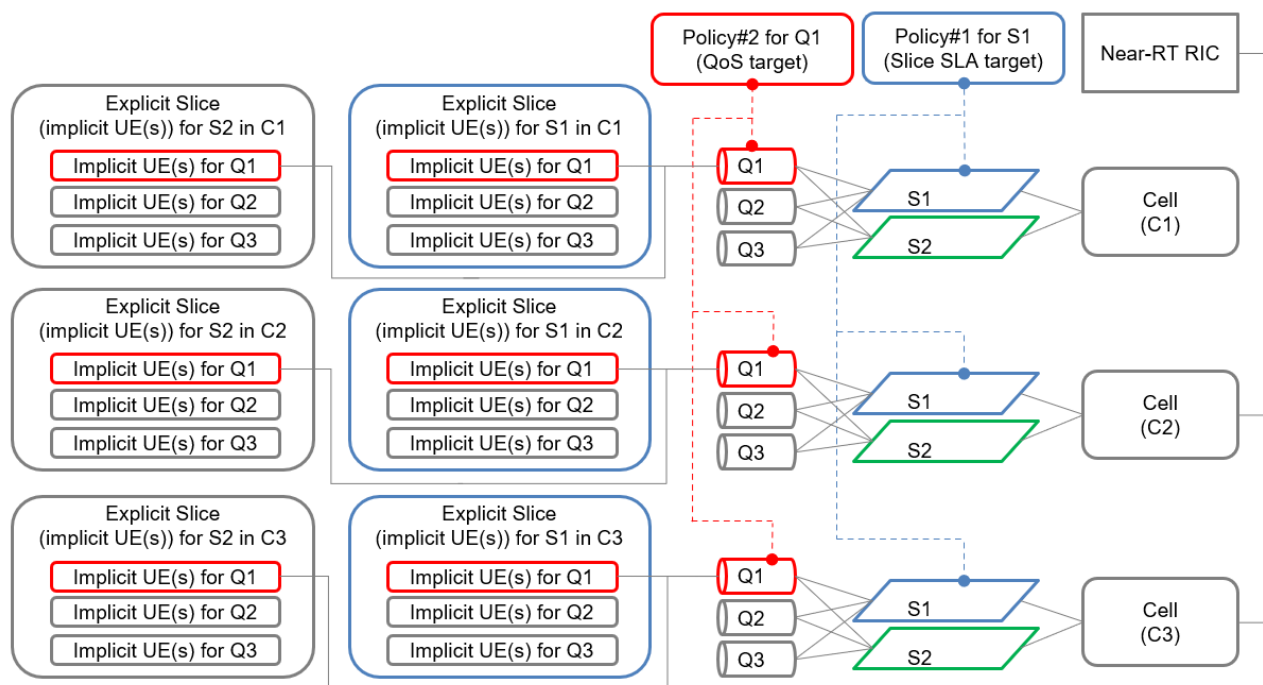


Figure 6.2.1- 4: An example of a PAET pair with explicit Slice and implicit UE(s) in case of Application-based slice

In Figure 6.2.1-5, another example of the set relation of PAET#1 with explicit Slice for Policy#1 and PAET#2 with implicit UE(s) for Policy#2 with an overlap is shown. In this example, three cells (C1, C2 and C3) are controlled by the Near-RT RIC, two different QoS-based slices (S1 and S2) per cell are configured, multiple QoS flows (Qx1...Qxn,

Qy1...Qyn) are uniquely handled in slices as shown in right side of Figure 6.2.1-3, and two policies (one for Slice SLA policy for slice S1 and the other for QoS target for QoS Qx1) are applied. As referenced from Table 6.2.1-2, within each of the explicit cells (C1, C2 and C3), the set of entities for PAET#1 is superset of PAET#2, and its overlap is equal to the set of entities for PAET#2.

6.2.2 Example criteria for finding overlaps with policy scope or policy resource.

There would be possibly four pairs of A1 policy applicable entity types (PAETs) between two policies which need to be compared and analysed for detecting overlaps. The structure of comparison and analysis of PAETs between two policies for detecting overlaps are illustrated in Figure 6.2.2-1.

In this figure, PAET1 is an A1 policy applicable entity type which is derived from attributes of scopeIdentifier, and PAET2 is an A1 applicable entity type which is derived from cell-identified attributes of policy resources. Numbers in the dotted circle in the Figure 6.2.2-1 indicate cases of comparison and analysis for a pair of PAETs: case 1 for a pair of two PAET1s, case 2 for a pair of two PAET2s, case 3 and case 4 for a pair of one PAET1 and another PAET2.]

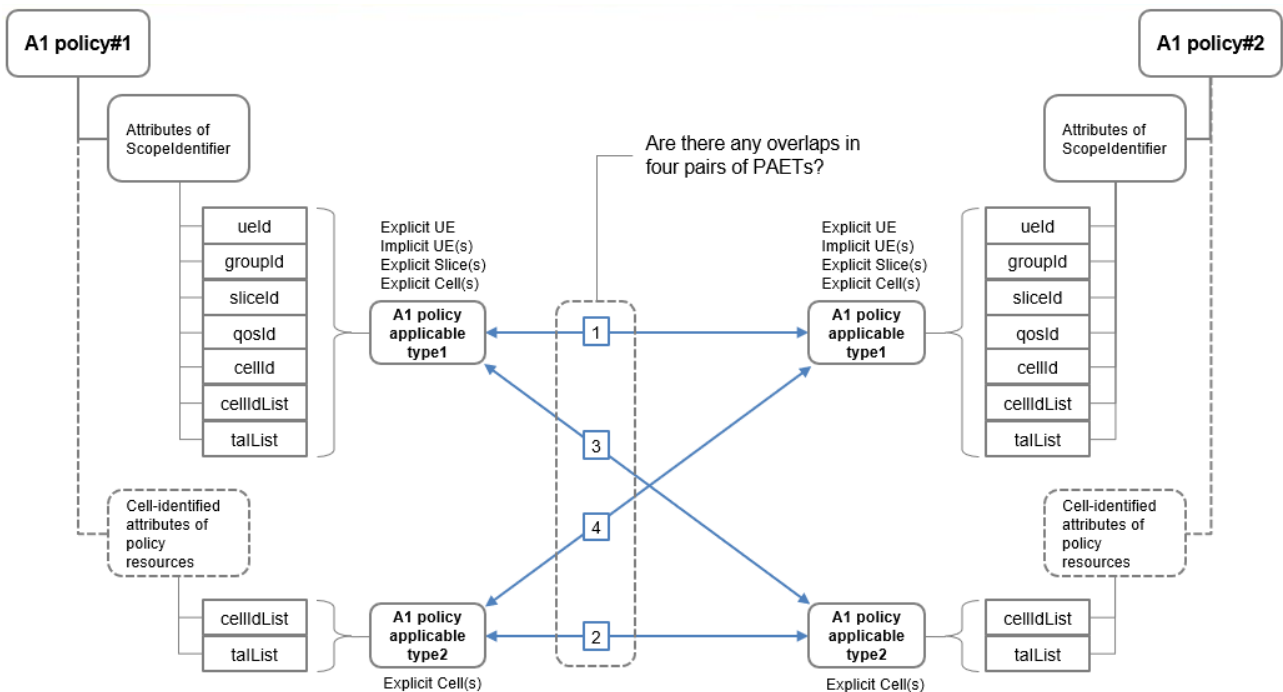


Figure 6.2.2- 1: Structure of comparison and analysis between two policies for detecting overlaps.

And as Table 6.2.1-2 shows, it is suggested for simplicity that comparison and analysis for a pair of PAETs to find overlaps would be performed cell-by-cell for all cells which a Near-RT RIC controls. Set relations of an overlap would be simply identified per specific cell rather than identifying the set relation with possible multiple cells simultaneously.

In Figure 6.2.2-2, an example of the set relation of PAET#1 with explicit Slice for Policy#1 and PAET#2 with explicit Cell for Policy#2 with an overlap is shown. In this example, three cells (C1, C2 and C3) are controlled by the Near-RT RIC, three different slices (S1, S2 and S3) per cell are configured, and two policies (one for Slice SLA policy for slice S1 and the other for Energy Savings for cell C2) are applied. As shown in right side of the figure, comparison and analysis for a pair of PAETs would be performed cell-by-cell for all cells which a Near-RT RIC controls. In this example, an overlap in cell C2 would be detected while there are no overlaps in cell C1 and C3.

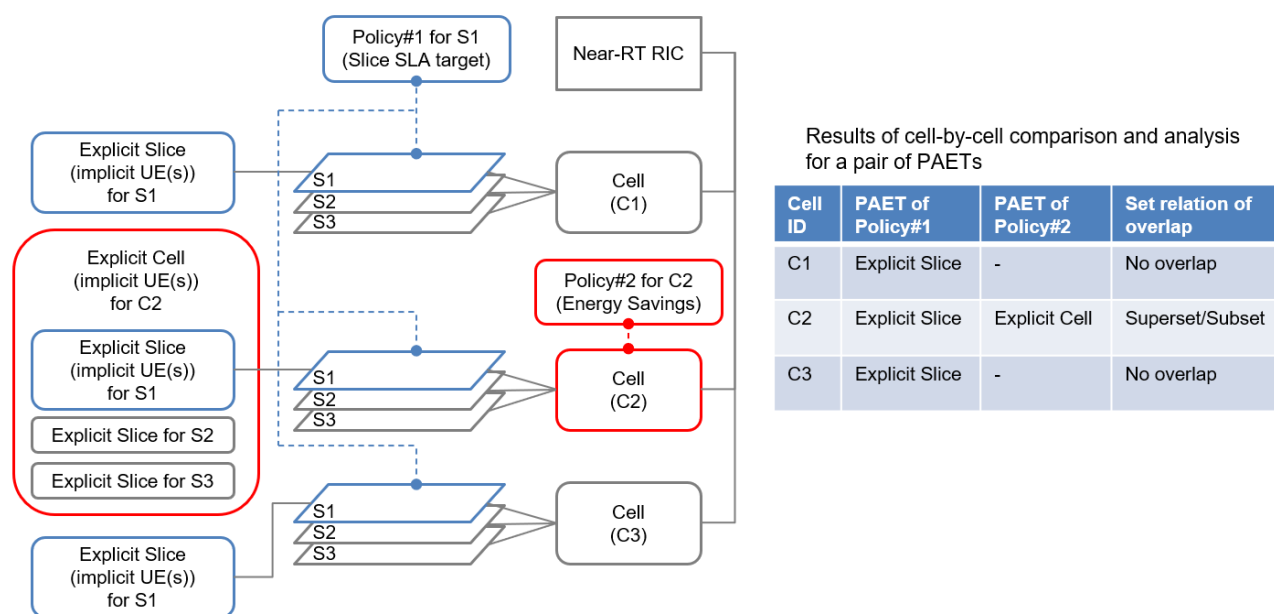


Figure 6.2.2- 2: An example of a PAET pair with explicit Slice and explicit Cell and its result of finding overlaps.

Criteria for finding overlaps with each pair of PAETs in set relations of policy scopes or policy resources per cell are shown.

< Case 1: a pair of two PAET1s >

- Finding a pair of two PAET1s with “identical” set relation of policy scopes would be to check if values of their ScopeIdentifiers are completely the same.
- Finding a pair of PAET1s with “subset/superset” relation of policy scopes would be to check if following conditions are met.
 - Either or both of ueId attributes in the scopeIdentifier are not specified.
 - One or more attributes in groupId, sliceId, qosId, cellId, cellIdList and taList are included in the scopeIdentifier.
 - The policies have at least one identical attribute value in their scopeIdentifiers.
 - Subset in policy scope generally has more non-identical attributes specified than superset in policy scope.
- Finding a pair of PAET1s with “joint sets” relation of policy scopes would be to check if following conditions are met.
 - Either or both of ueId attributes in the scopeIdentifier are not specified.
 - One or more attributes in groupId, sliceId, qosId, cellId, cellIdList and taList are included in the scopeIdentifier.
 - The policies do not have any identical attribute value in their scopeIdentifiers.
 - There is a non-empty intersection between two non-identical policies.

< Case 2: a pair of two PAET2s >

- Finding a pair of two PAET2s with “identical” set relation of policy resources would be to check if values of cell-identified attributes of policy resources are completely the same.
- Finding a pair of PAET2s with “subset/superset” relation of policy resources would be to check if following conditions are met.
 - A non-identical and non-empty superset PAET2 has all values of cell-identified attributes of policy resources of another subset PAET2.
- Finding a pair of PAET2s with “joint sets” relation of policy scopes would be to check if following conditions are met.
 - Non-identical and non-empty PAET2s have identical values for some but not all of the cell-identified attributes of policy resources of another PAET2.

< Case 3 and case 4: a pair of one PAET1 and another PAET2 >

- Finding a pair of one PAET1 and another PAET2 with “identical” set relation of policy scope and policy resource would be to check if values of one ScopeIdentifier and another cell-identified attributes of policy resource are completely the same.
- Finding a pair of one PAET1 and another PAET2 with “subset/superset” relation of policy scope and policy resource would be to check if following conditions are met.
 - ueId attribute in the scopeIdentifier of the PAET1 is not specified.
 - One or more attributes in groupId, sliceId, qosId, cellId, cellIdList and taList of the PAET1 are included in the scopeIdentifier.
 - The policies have at least one identical cell-identified attribute value in the scopeIdentifier of the PAET1 and policy resource of the PAET2.
 - Subset PAET generally has more non-identical attributes specified than superset PAET in policy scope or policy resource.
- Finding a pair of one PAET1 and another PAET2 with “joint sets” relation of policy scope and policy resource would be to check if following conditions are met.
 - ueId attribute in the scopeIdentifier of the PAET1 is not specified.
 - One or more attributes in groupId, sliceId, qosId, cellId, cellIdList and taList of the PAET1 are included in the scopeIdentifier.
 - The policies do not have any identical attribute value in their scopeIdentifiers.
 - There is a non-empty intersection between two non-identical policies.

6.2.3 Examples of overlap in policy scope

Some examples of finding overlaps between two policies in policy scope are shown below. In examples of this clause, two cells (C1 and C2) are controlled by a Near-RT RIC, two application-based slices (S1 and S2) are configured in each cell, three QoS flows (Q1, Q2 and Q3) are handled in slices, and two groupIds (referred as G1, G2 in the figures) with different spId(s) or rfspIndex(es) are configured. And as explained earlier, a pair of policies is analysed and compared cell-by-cell basis for all cells which a Near-RT RIC controls.

- Example1: a pair of Policy#1 with sliceId and qosId values (e.g., QoS/QoE target or TSP policy type as implicit UE(s)) and Policy#2 with sliceId value (e.g., Slice SLA target or QoE target policy type as implicit UE(s)), and their sliceId values would be identical.

Analysis and comparison for this example1 pair of policies are illustrated in Figure 6.2.3-1. For each of cell C1 and cell C2, it would be found that policy scope of Policy#1 is subset of policy scope of Policy#2 as matching the criteria of second bullet in case1 in clause 6.2.2.

Policy	ueld	groupid	sliceld	qosld	celld
Policy#1	0	0	(S1)	(Q1)	0
Policy#2	0	0	(S1)	0	0

↓

Policy	ueld	groupid	sliceld	qosld	celld	Set relation of overlap
Policy#1	0	0	(S1)	(Q1)	(C1)	Subset
Policy#2	0	0	(S1)	0	(C1)	Superset

Policy	ueld	groupid	sliceld	qosld	celld	Set relation of overlap
Policy#1	0	0	(S1)	(Q1)	(C2)	Subset
Policy#2	0	0	(S1)	0	(C2)	Superset

Figure 6.2.3- 1: Analysis and comparison for finding overlap in example1.

The applicable entities relation with overlap (parts filled in yellow in referred figure) between two policies in example1 are illustrated in Figure 6.2.3-2. In the referred figure, there are four separated sets of applicable entities as implicit UE(s) for Policy#1 (red-line circle parts) and four separated sets of applicable entities as implicit UE(s) for Policy#2 (blue-line circle parts) which would differ based on combination of cell (C1 or C2) and DRB (G1 or G2).

In each set of applicable entities for Policy#2, there is a subset of entities for QoS flow Q1 for Policy#1, and those four subsets are overlapping sets between two policies.

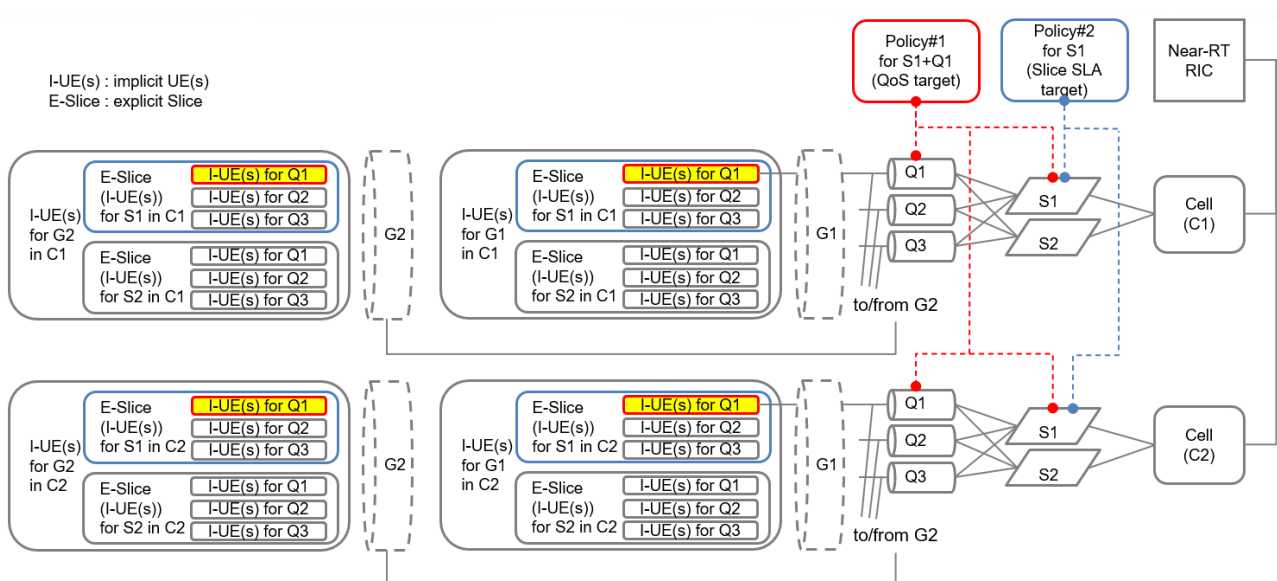


Figure 6.2.3- 2: Illustration of applicable entities relation with overlap between two A1 policies in example1.

- Example2: a pair of Policy#1 with sliceId and qosId values (e.g., QoS/QoE target or TSP policy type as implicit UE(s)) and Policy#2 with qosId value (e.g., QoE target policy type as implicit UE(s)), and their qosId values would be identical.

Analysis and comparison for this example2 pair of policies are illustrated in Figure 6.2.3-3. For each of cell C1 and cell C2, it would be found that policy scope of Policy#1 is subset of policy scope of Policy#2 as matching the criteria of second bullet in case1 in clause 6.2.2.

Policy	ueld	groupid	sliceId	qosId	cellId
Policy#1	0	0	(S1)	(Q1)	0
Policy#2	0	0	0	(Q1)	0

↓

Policy	ueld	groupid	sliceId	qosId	cellId	Set relation of overlap
Policy#1	0	0	(S1)	(Q1)	(C1)	Subset
Policy#2	0	0	0	(Q1)	(C1)	Superset

Policy	ueld	groupid	sliceId	qosId	cellId	Set relation of overlap
Policy#1	0	0	(S1)	(Q1)	(C2)	Subset
Policy#2	0	0	0	(Q1)	(C2)	Superset



Figure 6.2.3- 3: Analysis and comparison for finding overlap in example2.

The applicable entities relation with overlap (yellow part in referred figure) between two policies in example2 are illustrated in Figure 6.2.3-4. In the referred figure, there are four separated sets of applicable entities as implicit UE(s) for Policy#1 (red-line circle parts) which would differ based on combination of cell (C1 or C2) and DRB (G1 or G2), and eight separated sets of applicable entities as implicit UE(s) for Policy#2 (blue-line circle parts) which would differ based on combination of cell (C1 or C2), DRB (G1 or G2) and slice (S1 or S2).

Each of all four separated sets for Policy#1 are identical to one of eight separated sets for Policy#2, and those four subsets are overlapping sets between two policies. It is noted that all overlapping sets (yellow part in referred figure) are drawn in red-line, but they should also be drawn in blue-line because those sets are identical in both policies.

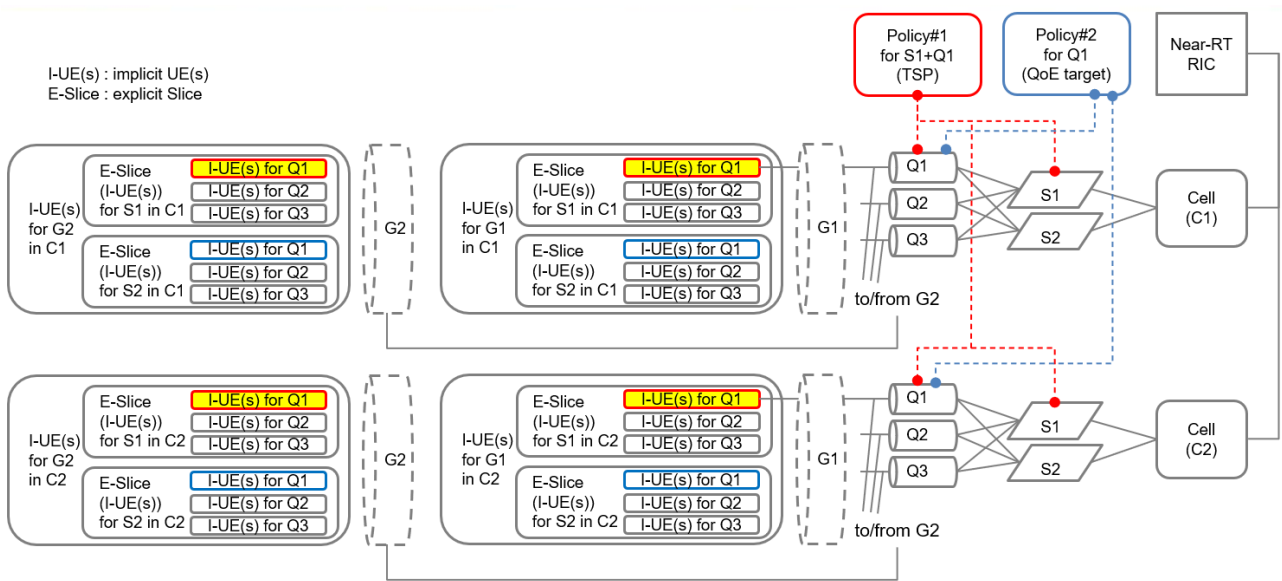


Figure 6.2.3- 4: Illustration of applicable entities relation with overlap between two A1 policies in example2.

- Example3: a pair of Policy#1 with groupId and qosId values (e.g., QoS target policy type as implicit UE(s)) and Policy#2 with sliceId value (e.g., Slice SLA target or QoE target policy type as implicit UE(s)).

Analysis and comparison for this example3 pair of policies are illustrated in Figure 6.2.3-5. For each of cell C1 and cell C2, it is found that policy scopes of both Policy#1 and Policy#2 are joint sets as matching the criteria of third bullet in case1 in clause 6.2.2.

Policy	ueld	groupid	sliceId	qosId	cellId
Policy#1	0	(G1)	0	(Q1)	0
Policy#2	0	0	(S1)	0	0

Policy	ueld	groupid	sliceId	qosId	cellId	Set relation of overlap
Policy#1	0	(G1)	0	(Q1)	(C1)	Jointset
Policy#2	0	0	(S1)	0	(C1)	Jointset

Policy	ueld	groupid	sliceId	qosId	cellId	Set relation of overlap
Policy#1	0	(G1)	0	(Q1)	(C2)	Jointset
Policy#2	0	0	(S1)	0	(C2)	Jointset

Figure 6.2.3- 5: Analysis and comparison for finding overlap in example3.

The applicable entities relation with overlap (yellow part in referred figure) between two policies in example3 are illustrated in Figure 6.2.3-6. In the referred figure, there are four separated sets of applicable entities as implicit UE(s)

for Policy#1 (red-line circle parts) which would differ based on combination of cell (C1 or C2) and slice (S1 or S2), and four separated sets of applicable entities as implicit UE(s) for Policy#2 (blue-line circle parts) which would differ based on combination of cell (C1 or C2) and DRB (G1 or G2).

In two sets of four separated sets of applicable entities for Policy#2 which are identified by combination of slice S1 and DRB G1, there is a subset of entities for QoS flow Q1 for Policy#1, and those two subsets out of four subsets are overlapping sets between two policies.

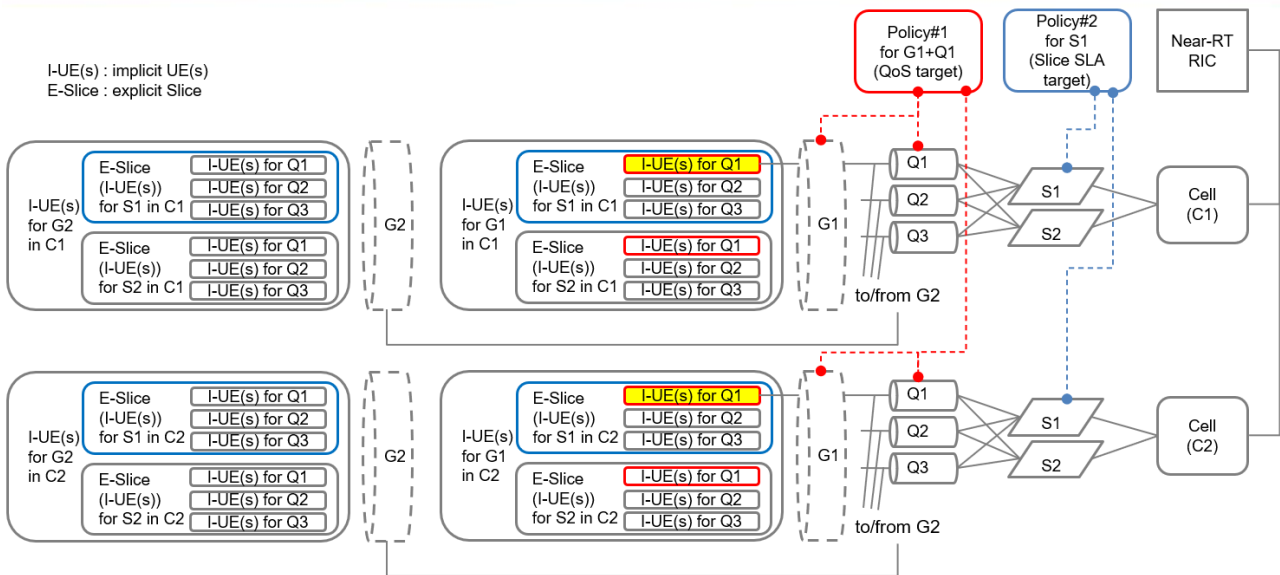


Figure 6.2.3- 6: Illustration of applicable entities relation with overlap between two A1 policies in example3.

- Example4: a pair of Policy#1 with qosId value (e.g. QoS/QoE target as implicit UE(s)) and Policy#2 with cellId value (e.g., LB or ES policy type as explicit Cell).

Analysis and comparison for this example4 pair of policies are illustrated in Figure 6.2.3-7. For cell C1, it is found that policy scope of Policy#1 is subset of policy scope of Policy#2 as matching the criteria of second bullet in case1 in clause 6.2.2. For cell C2, policy scope of Policy#2 is not applicable and there is not overlap between two policies.

Policy	ueld	groupid	sliceld	qosld	cellld
Policy#1	0	0	0	(Q1)	0
Policy#2	0	0	0	0	(C1)

↓

Policy	ueld	groupid	sliceld	qosld	cellld	Set relation of overlap
Policy#1	0	0	0	(Q1)	(C1)	Subset
Policy#2	0	0	0	0	(C1)	Superset

Policy	ueld	groupid	sliceld	qosld	cellld	Set relation of overlap
Policy#1	0	0	0	(Q1)	(C2)	No overlap
Policy#2	0	0	0	0	(C2)	

Figure 6.2.3- 7: Analysis and comparison for finding overlap in example4.

The applicable entities relation with overlap (yellow part in referred figure) between two policies in example4 are illustrated in Figure 6.2.3-8. In the referred figure, there are eight separated sets of applicable entities as implicit UE(s) for Policy#1 (red-line circle parts) which would differ based on combination of cell (C1 or C2), slice (S1 or S2) and DRB (G1 or G2), and two separated sets of applicable entities as implicit UE(s) for Policy#2 (blue-line circle parts) which would differ based on DRB (G1 or G2).

In each set of two separated sets of applicable entities for Policy#2, there is two subsets of entities for QoS flow Q1 for Policy#1, and those four subsets out of eight subsets are overlapping sets between two policies.

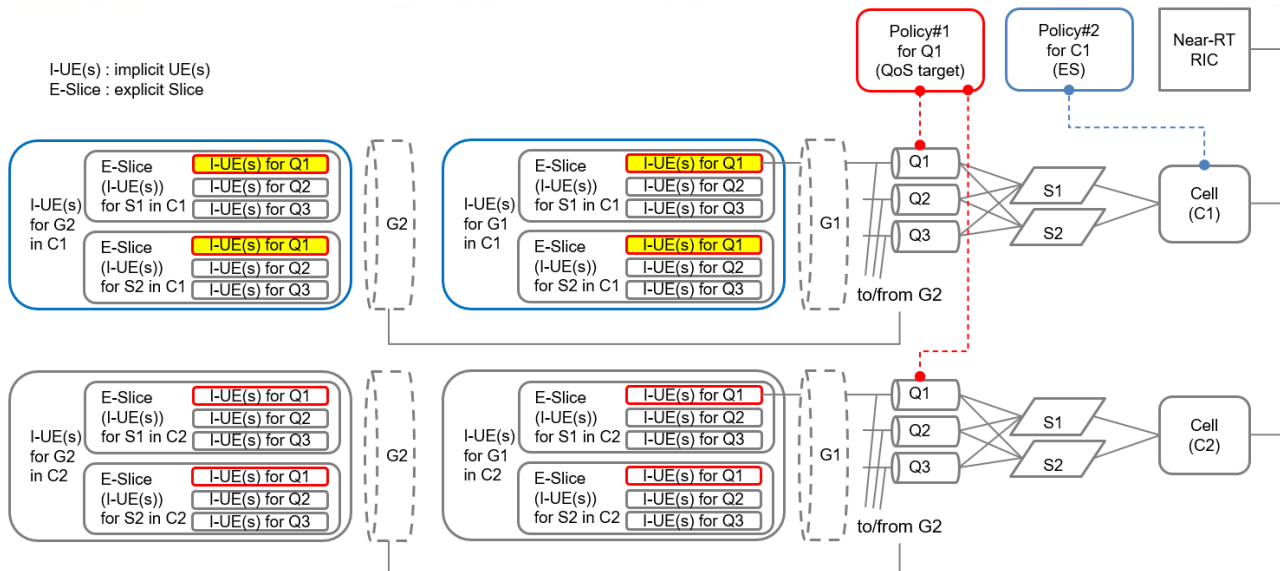


Figure 6.2.3- 8: Illustration of applicable entities relation with overlap between two A1 policies in example4.

6.2.4 Overlap information of A1 policy scopes and policy resources

As a result of analysing the A1 policy scopes and policy resource with a set of A1 policies for a creation/update requested policy, information regarding overlap is useful and necessary to find alternative conflict-free policies. Overlap information of A1 policy scopes and policy resources is the information which describes how policy scopes or policy resources are overlapping. Contents of the overlap information would be suggested as follows:

Detected overlap information per a requested A1 policy contains a list of followings:

- PolicyId of requested A1 policy.
- List of PAET pairs with per-cell-basis (cellId) overlap between requested policy and existing policy (PolicyId)
 1. whether the overlap is between policy scopes
 2. whether the overlap is between requested policy scope and policy resource of existing policy
 3. whether the overlap is between requested policy resource and policy scope of existing policy
 4. whether the overlap is between policy resources

Type of overlap (Identical/Subset/Superset/Joint set) for a specific cell with a pair of PAETs

6.3 Detection of conflicting situations

The detection of conflicting situations starts with finding a set of overlapping pairs of policies. Policy scope and/or policy resource of a requested policy is analysed with policy scope and/or policy resource of an existing policy to detect an overlapping pair of requested policy and existing policy. This overlap detection process is repeated over all the pairs of requested and existing policies to arrive at a set of overlapping pairs of policies. According to figure 6.1-1, once overlaps with policy scope and/or policy resource are detected among A1 policies, it would be evaluated per overlap if there are any conflicting situations with policy objectives and/or policy resources to finally recognize a set of overlapping policies as conflicting policies.

6.3.1 Evaluation of conflicting situation for collection of overlaps

As explained in clause 6.2, detection of overlap would be performed on a cell-by-cell basis for simplicity across all cells which a Near-RT RIC controls. And the evaluation of conflicting situation would be performed for collection of detected overlaps.

Given that multiple Near-RT RICs are connected with an SMO/Non-RT RIC, there would be groups of overlaps per Near-RT RIC which would be divided into groups per cell. An example of grouping of detected overlaps is illustrated in Figure 6.3.1-1 in which detected overlaps are indexed by p-q-r where p is the index of Near-RT RIC, q the index of cell and r the index of detected overlap uniquely for that cell, along with corresponding sets of overlapping policies.

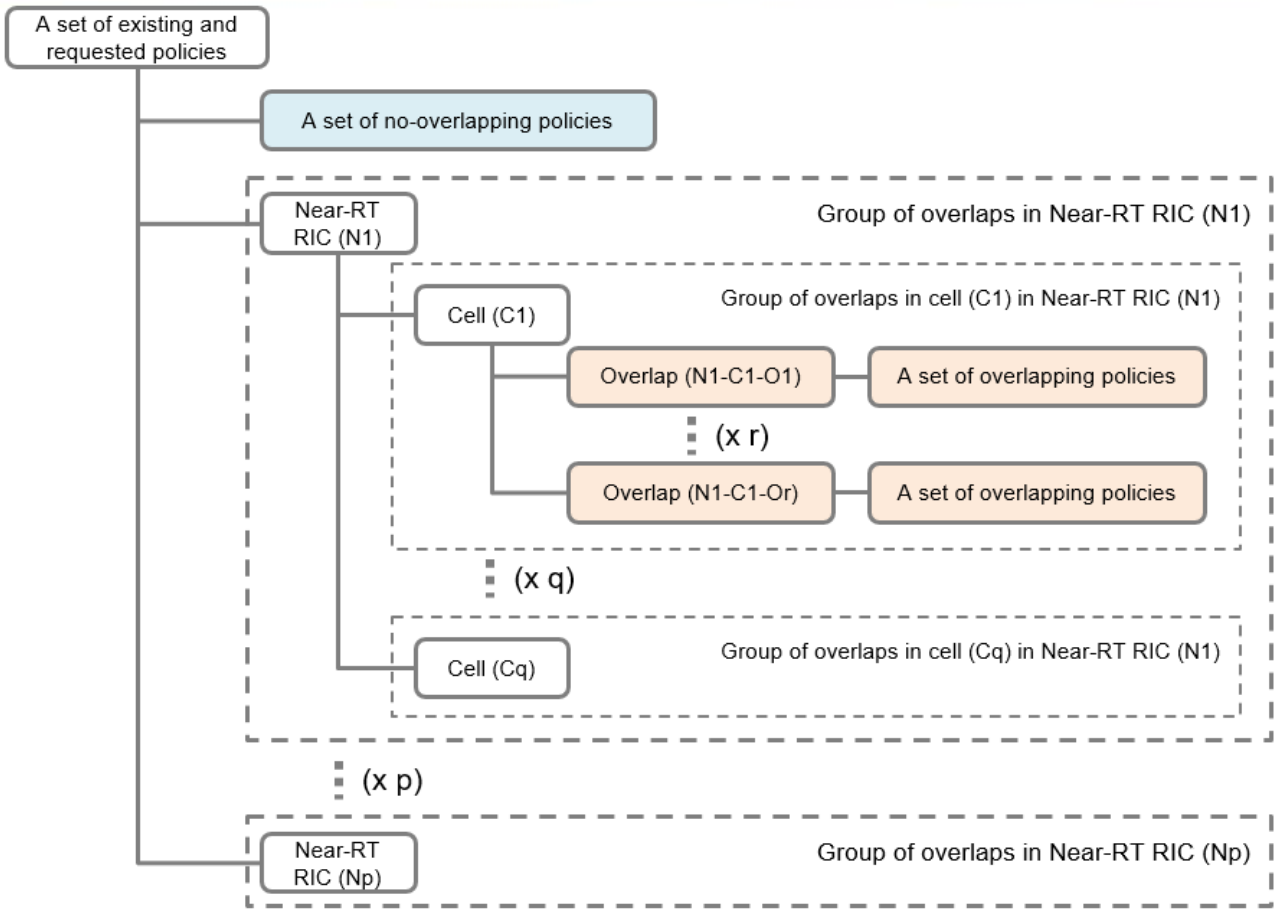


Figure 6.3.1- 1: An example of grouping of detected overlaps.

It is noted that there would be cases that the same A1 policy belongs to different detected overlaps concurrently.

Considering the flexibility of related functional placement in the SMO/Non-RT RIC, the uniqueness of complete information regarding detected conflicts in the SMO/Non-RT RIC would be necessary. For example, in a case where SMO/Non-RT RIC FW is the producer of A1 policy conflict detection service and an rApp is the consumer of the A1 policy conflict detection service to perform conflict avoidance, information related to a detected conflict needs to be uniquely conveyed over the R1.

6.3.2 Approach to evaluate conflicting situation.

Possible conflicting situations have been studied in clause 5 Use cases with representative scenarios. There would be possibly various approaches to evaluate conflicting situation with use of such as the AI/ML or Digital twin, and related discussion regarding those detailed algorithm would be left for further investigation, however, here it is suggested that basic approach to evaluate conflicting situation is to check if pairs of policy scope and/or policy resource with overlap between two policies would meet those A1 policy conflict use cases; UC#1 “reserve” vs “release” policy, UC#2 objectives not uniquely identified, and UC#3 conditions not uniquely identified.

Possible A1 policy conflict use cases could be related into pairs of policy scopes and/or policy resources with a detected overlap as shown in Figure 6.3.2-1. There would possibly be four pairs of policy scopes and/or policy resources having an overlap between a requested policy and one of existing policies, and for each of detected overlaps between two policies, it would be checked if corresponding A1 policy conflict use cases would meet or not. If a requested policy has at least one conflicting situation to meet those A1 policy conflict use cases for detected overlaps, a set of detected

overlapping policies would be regarded as a set of conflicting policies and the requested policy would be a member of conflicting policies.

A pair of policy scopes and/or policies resources with a detected overlap	Existing policy scope	Existing policy resource
Requested policy scope	#1 UC#1. "reserve" vs "release" policy UC#2. objectives not uniquely identified	#2 UC#1. "reserve" vs "release" policy
Requested policy resource	#3 UC#1. "reserve" vs "release" policy	#4 UC#1. "reserve" vs "release" policy UC#3. conditions not uniquely identified

Figure 6.3.2- 1: Mapping of possible A1 policy conflict use cases into a pair of policy scopes and/or policy resources with a detected overlap.

It is noted that a pair of policy scopes and/or policy resource with a detected overlap would possibly meet multiple conflicting situations as shown in #1 and #4 pairs in Figure 6.3.2-1. An example of detected overlap (O1) in cell (C1) under the control of Near-RT RIC (N1) with a result of checking conflicting situations is illustrated in Figure 6.3.2-2. In the referred figure, a set of overlapping policies could be divided into a set of conflict-free policies and multiple sets of conflicting policies.

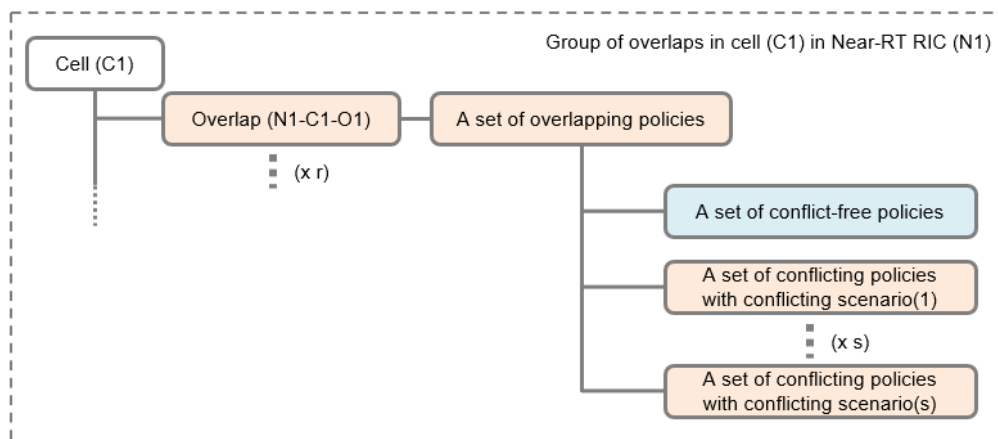


Figure 6.3.2- 2: Group of overlaps with results of checking conflicting situations.

It is also noted that contents of A1 policies such as policy type, policy objective and policy resource would be typically used to find conflicting situations for A1 policy conflict use cases, and its details for some algorithms are left to implementation. It might be considered that available UE-related attributes (e.g. UE context) are used to recognize the actual existence of implicit UE(s) for conflicting policies (e.g., for evaluating the severity of impact of conflicting situations with such as a number of affected UEs), however, it should be noted that this condition would be intermittent and not persistent considering possible UE handovers and its time lag between collection and use of UE-related attributes in the RAN. The use of UE-related attributes for evaluation of conflicting situations would be also left to implementation.

6.3.3 Information of conflicting situations

As a result of evaluating conflicting situations for sets of overlapping policies, some sets of conflicting policies might be found with information of conflicting situations such as matchings of A1 policy conflict use cases. That information of conflicting situations among conflicting policies would be useful and necessary for conflict mitigations, such as providing different priorities to conflicting policies for A1 policy conflict use case #1 or aligning different values of policy objectives for A1 policy conflict use case #2.

6.4 A1 policy conflict detection service

A1 policy conflict detection service optionally takes A1 policies, for which conflicts need to be detected, as input and provides overlap information (clause 6.2.4) and information of conflicting situation (clause 6.3.3) for detected sets of potential conflicting policies as output, regardless of the enforcement status (ex. enforced or not_enforced) of all existing policies. The service is triggered and its output is used by the A1 policy conflict detection service consumer for the purpose of policy conflict avoidance.

The A1 policy conflict detection service producer and the A1 policy conflict detection service consumer could be either the SMO/Non-RT RIC FW or an rApp.

The use cases of the A1 policy conflict detection service with inputs from the service consumer are listed in Table 6.4-1; one is for creating a new policy against all existing policies (use case 2 in Table 6.4-1), and another is for modifying an existing policy against all existing policies (use case 3 in Table 6.4-1). The service could be utilized either before creating or updating an A1 policy to check its feasibility, or after A1 policies have already been created or updated by rApps in Near-RT RICs (use case 1A/1B in Table 6.4-1). The service should also support a method to verify its conflict detection algorithm (use case 4 in Table 6.4-1). It should be noted that A1 policy conflict detection service producer needs all information about existing A1 policies for conflict detection, however, the A1 conflict detection service consumer would typically provide one or multiple A1 policies for the service consumer to create or modify by itself. If it would be a case that all existing A1 policies need to be considered together for conflict detection, they would need to be provided from A1 policy management services as defined in R1GAP [i.3], clause 5.3.2.

Table 6.4- 1: Use cases of the A1 policy conflict detection service with inputs from the service consumer.

Use cases	Policies provided from the service consumer	Indicator to subscribe for conflict detection notification	Indication about whether to find conflicts with existing policies	Identifier for the existing policies to be excluded from the conflict detection
1A. Conflict check for existing policies on request basis	No	No	Yes	No
1B. Conflict check for existing policies on periodical scan basis	No	Yes	Yes	No
2. Conflict check for both a new policy (Policy-A) and existing policies	Yes (Policy-A)	No	Yes	No
3. Conflict check for both a modified existing policy (Policy-B') and existing policies except the policy (Policy-B) to be modified	Yes (Policy-B')	No	Yes	Yes (Policy-B)
4. Conflict check for provided policies (Policies-X1, ..., Xn) for verifying the conflict detection algorithm	Yes (Policies-X1, ..., Xn)	No	No	No

It is noted that the A1 policy functions in SMO/Non-RT RIC FW could be the conflict detection service consumer as needed when it receives the A1 policy creation/modification request.

One of possible implementation approaches for A1 policy management services to provide information about other existing A1 policies to the A1 policy conflict detection service producer would be for A1 policy management service producer to keep track of the existence of A1 policies transferred over A1 interface and cache them inside, and to provide them from the cache. The ability for A1 policy management service producer to keep track of the existence of A1 policies would reduce the workload of collection for existing policies over A1 interface as well as the workload of Near-RT RICs, comparing to always querying existing A1 policies which would invoke multiple transactions over A1 interface with Near-RT RICs.

The sequence diagram of the A1 policy conflict detection service on request basis is illustrated in Figure 6.4-1. In the referred sequence diagram, as part of the conflict detection request, the A1 policy conflict detection service customer would provide the A1 policies for which the conflicts need to be detected, indication about whether to find conflicts with existing policies, and the identifier for the existing policies to be excluded from the conflict detection. In the case of updating an existing A1 policy, A1 policy conflict detection service customer can use the identifier for the existing policy to exclude it from the conflict detection.

```
@startuml
Skin rose
Skinparam ParticipantPadding 5
Skinparam BoxPadding 10
skinparam defaultFontSize 12
```

```

autonumber

Box "SMO/Non-RT RIC" #gold

Participant " A1 policy conflict detection service \nConsumer" as a1
Participant " A1 policy conflict detection service \nProducer" as arb
Database " A1 policy management services \nProducer " as db

End box

a1->arb:A1 policy conflict detection request\n<color #green>(A1 policies for conflict
detection, \n<color #green>Indication about whether to find conflicts with existing
policies, \n<color #green>Identifier for the existing policies to be excluded from the
conflict detection)

|||

Opt a case of using existing policies together for conflict detection
    arb->db:query existing A1 policies information request
    |||
    db->arb:query existing A1 policies information response\n<color #green>(information
of existing A1 policies)
    |||
    Opt a case of excluding a specified policy for conflict detection
        arb->arb:excluding a specified policy for conflict detection
    End
End

arb->arb:performing conflict detection

arb->a1:A1 policy conflict detection response \n<color #green>(overlap information,
information of conflicting situation)

@enduml

```

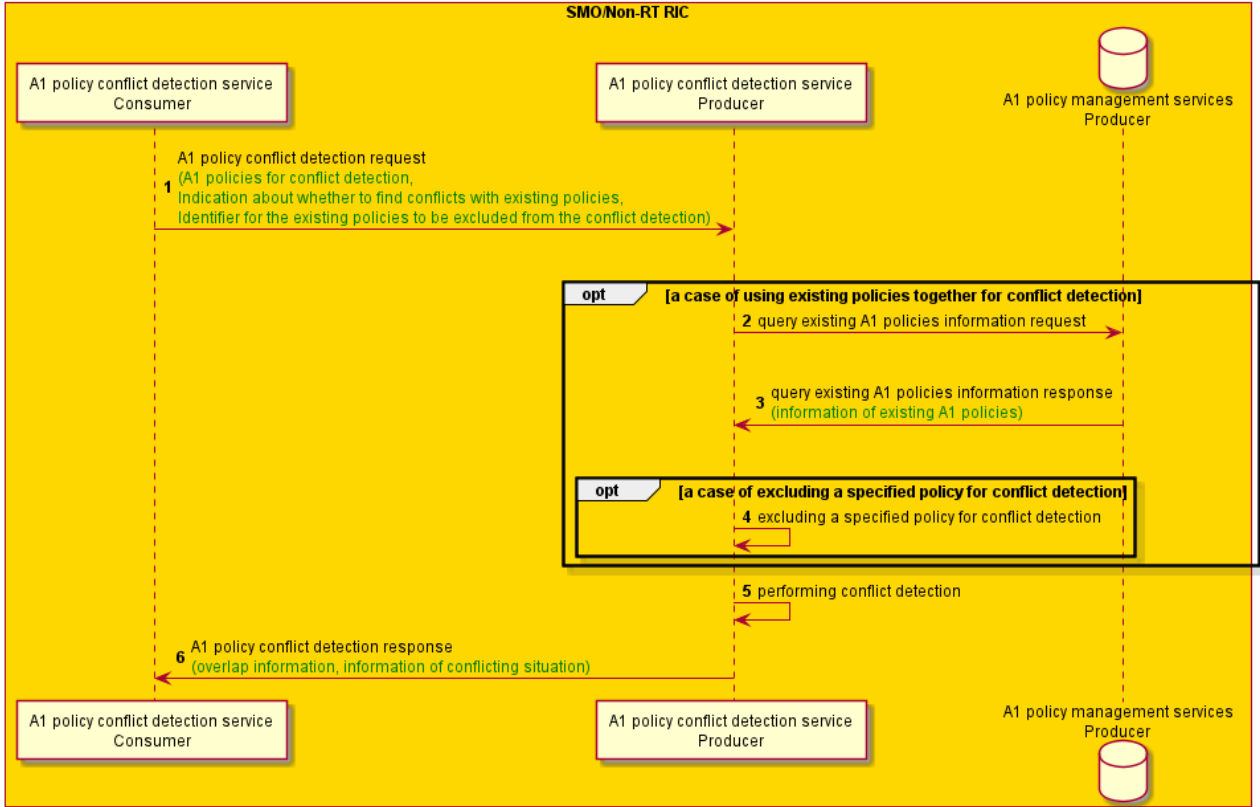


Figure 6.4- 1: A1 policy conflict detection service on request basis

The sequence diagram of the A1 policy conflict detection service on subscription basis is illustrated in Figure 6.4-2. In the referred sequence diagram, the A1 policy conflict detection service customer would request to subscribe to the A1 policy conflict detection notification and would get the A1 policy conflict detection notification if detected by the periodical scan of the A1 policy conflict for existing A1 policies.

```
@startuml
Skin rose
Skinparam ParticipantPadding 5
Skinparam BoxPadding 10
skinparam defaultFontSize 12
autonumber
Box "SMO/Non-RT RIC" #gold
Participant " A1 policy conflict detection service \nConsumer" as a1
Participant " A1 policy conflict detection service \nProducer" as arb
Database " A1 policy management services \nProducer " as db
End box
a1->>arb:Request to subscribe to A1 policy conflict detection notification
arb-->a1:Response for subscription
|||
```

```

loop periodical scan of A1 policy conflict

    arb->db:query existing A1 policies information request

    |||

    db->arb:query existing A1 policies information response\n<color #green>(information
of existing A1 policies)

    |||

    arb->arb:performing conflict detection

    alt conflict detected

        arb->a1:A1 policy conflict detection response \n<color #green>(overlap
information, information of conflicting situation)

    end

end

@enduml

```

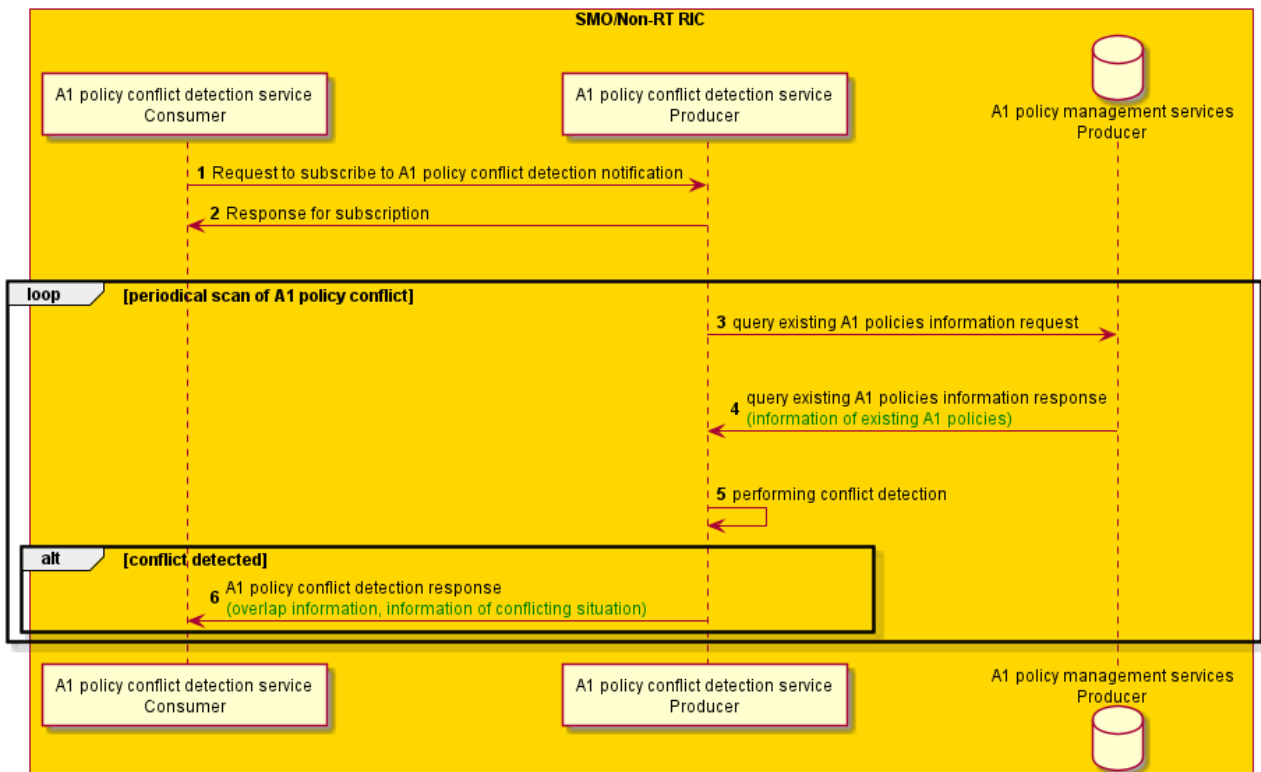


Figure 6.4- 2: A1 policy conflict detection service on subscription basis

7 A1 policy conflict avoidance

7.1 General

A1 policy conflict avoidance in SMO/Non-RT RIC aims to prevent the detected conflicts before the A1 policies that introduce conflicts with one or more existing policies are sent to Near-RT RIC. A potential A1 policy conflict can be detected by an rApp prior to sending a create or update A1 policy request or by the producer of the A1 policy

management service as part of processing the create or update A1 policy request. An rApp or the producer of A1 policy management service can detect potential A1 policy conflicts by themselves or by consuming the SMO service/R1 service that has the capability to detect potential A1 policy conflicts. Once a potential conflict detected, rApp or any other consumer of the A1 policy management services or the producer of the A1 policy management services can proceed with conflict avoidance by relying on conflict arbitration service.

The conflict arbitration service consumer consumes conflict arbitration service to determine the execution preference among a set of conflicting policies. Based on this, SMO/Non-RT RIC can further implement conflict avoidance procedures to prevent the conflicts.

One of the conflict avoidance strategies, among others, could be to reject a request for the creation or update of an A1 policy. Information regarding the conflicts details and the proposals for alternative A1 policies can be conveyed in reject message. As a follow-up action to the earlier conflict avoidance actions, an rApp can be notified to retry the creation of the previously rejected policy once the existing A1 policies causing the conflicts are deleted.

7.2 A1 policy conflict arbitration service

A1 policy conflict arbitration service provides A1 policy conflict arbitration rules as set by the operator or outside customer which then can be used by the consumer of the A1 policy conflict arbitration service to decide among two or more conflict avoidance actions when there is a potential A1 policy conflict. Similarly, A1 policy conflict arbitration service can also provide A1 policy conflict arbitration rules as set by the operator or outside customer which then can be used by the consumer of the A1 policy conflict arbitration service to decide among two or more conflict resolution or mitigation actions when there is an actual A1 policy conflict. A1 policy conflict arbitration rules help the consumer of the A1 policy conflict arbitration service to know the execution preference among the conflicting A1 policies. Conflict arbitration service producer can provide priorities for a set of conflicting policies, the A1 policies with priorities will be provided as follows:

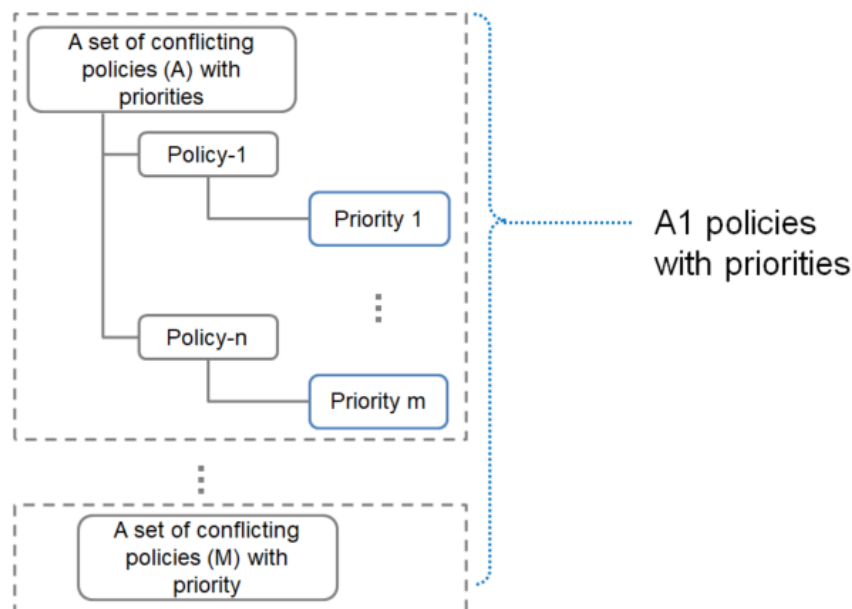


Figure 7.2- 1: A1 policies with priorities

In the following figures in this clause, the producer of SMO service/R1 Service with conflict arbitration capabilities can provide the service.

7.2.1 A1 policy conflict arbitration - Option #1. Handle in sequence

This A1 Policy conflict arbitration rule indicates to the consumer of the A1 policy conflict arbitration service that the requested A1 policy has lower priority while the existing policies have higher priority.

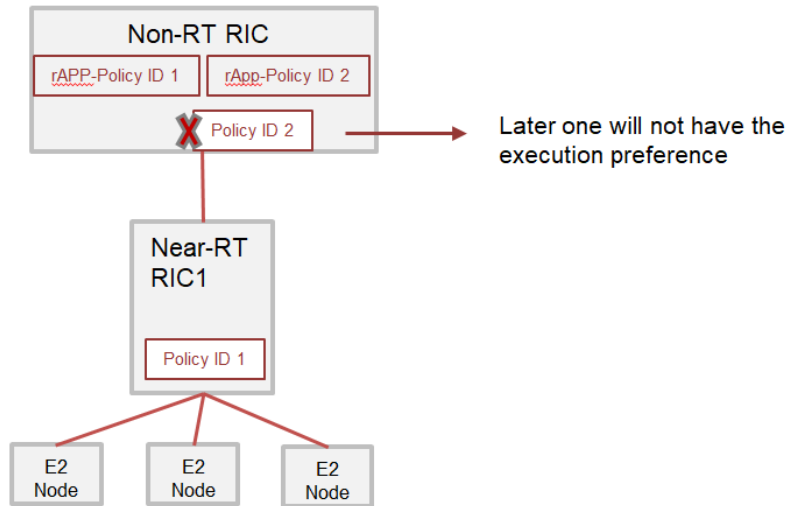


Figure 7.2.1- 1: Option #1 for conflict arbitration

7.2.1.1 Procedures

7.2.1.1.1 Handle in sequence

The figure below shows the message sequence of Option#1. Once a conflict is detected, A1 policy priorities base on the A1 policy creation/update time sequence. This option is not friendly for A1 policy service consumers to express their enforcement preferences.

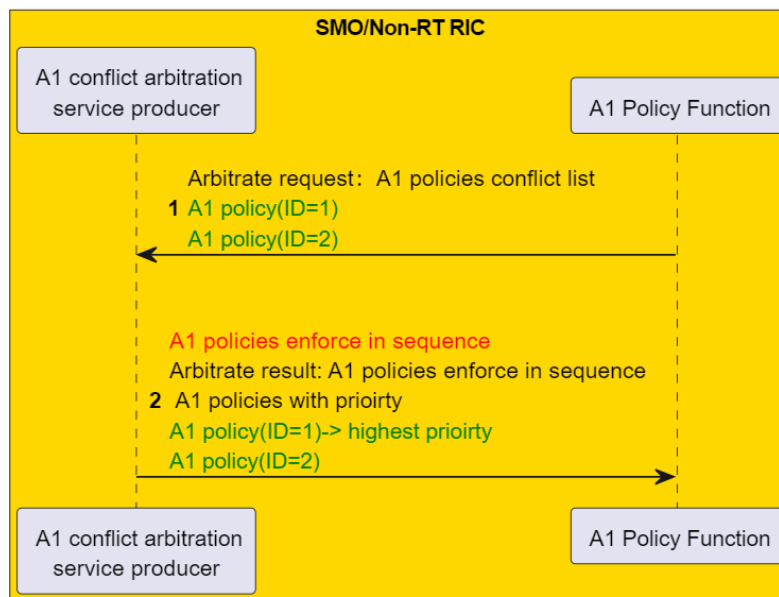


Figure 7.2.1.1.1- 1: Option #1 for conflict arbitration

7.2.2 A1 policy conflict arbitration - Option #2. Priority, Preemption protection

In Option#2, these A1 Policy conflict arbitration rules indicate the priority, preemption flags, and preemption protection flags of different policies to the consumer of A1 policy conflict arbitration service. Preemption Flag and Preemption Protection effectively address the needs of services that require immediate execution or cannot be interrupted once execution has begun.

Some guidelines for the arbitration rules regarding priority, the preemption flag, and the preemption protection flag are listed below:

- **Preemption Flag** indicate that, when a set of A1 policies conflict is detected, one of an A1 policy with this flag can pre-empt the execution of another A1 policy.
- **Preempt Protection Flag** indicates that when conflict is detected among a set of A1 policies, if one of the A1 policies with this flag is already being enforced, the execution of this A1 policy cannot be pre-empted by another A1 policy.
- **Priority** indicates the execution preference among a set of conflict A1 policies.

Some corner cases may also need to be considered in implementation. For example, both preemption flag and priority are present. Another example is a conflict is detected when two A1 policies share the same priority.

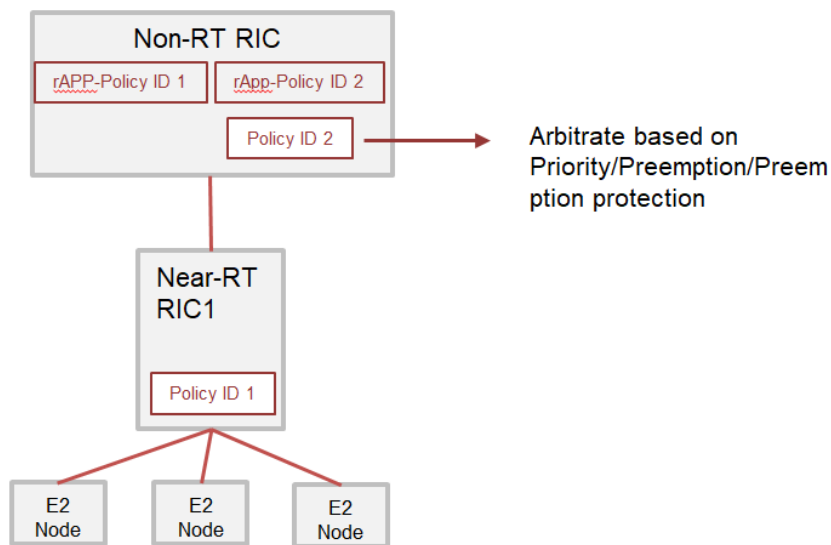


Figure 7.2.2- 1: Option #2 for conflict arbitration

7.2.2.1 Functionality analysis

7.2.2.1.1 A1 policy priority and pre-emption information

The operators can set the influencing factors of A1 policy priorities at relevant service producers who have the capabilities to provide the conflict arbitration rules via SMO service/R1 Service. Some influencing factors of A1 policy priorities are listed in Table 7.2.2.1-1. Operators or other possible service consumers can dynamically update these priorities in Table 7.2.2.1-1. A1 policy conflict arbitration service can then be consumed by different consumers such as rApps, A1 policy management function to decide execution preference among two or more conflict A1 policies. Conflict arbitration service producer can also determine A1 policy priority and pre-emption information. There can be

deployment scenarios where conflict arbitration service is produced by A1 policy management functions (Note: This approach fulfills operator's specific deployment requirement).

Table 7.2.2.1- 1: Influencing factors of A1 policy priority

Options	Contents	Notes
#1 rApp priority	In current stage, operators or service consumers who deploy the rApp can dynamically configure the priorities of rApps.	rApp priorities are not related to intents in current stage O-RAN specification.
#2 A1 Policy objectives & resource priorities	The priorities of A1 policies should reflect the intents that triggered these A1 policies.	One A1 policy may include several objectives and resource priorities.
#3 KPI targets priority representing operator's intent	A1 policy can also reflect the KPI targets which set by operators or service consumers. Non-RT RIC/SMO framework can set the priority base on KPI targets.	Current specifications do not specify how the KPI targets are associated with an A1 policy.

As for the A1 policy conflict use case #1 “reserve policy” vs. “release policy” as shown in clause 5.1, there would be some of potential solutions for operators to configure the A1 policy priority. Under the low traffic load condition where RAN resources are abundant for guaranteeing the service qualities, operators would prefer release policy over reserve policy, and hence operators would put higher priority for EsObjectives (as defined in A1TD [i.4], clause 6.3.3.7) than QosObjectives (as defined in A1TD [i.4], clause 6.3.3.2) with option.2 in Table 7.2.2.1-1. If these policies are owned by separate rApps, operators would put higher priority for rApp-X which owns policies of release policy group than rApp-Y which owns policies of reserved policy group with option.1 in Table 7.2.2.1-1. By contrast, under the high traffic load condition where securing RAN resources to guarantee the service qualities is becoming difficult, operators would prefer reserve policy over release policy, and hence operators would put higher priority for QosObjectives than EsObjectives with option.2 in Table 7.2.2.1-1, or operators would put higher priority for rApp-Y which owns policies of reserved policy group than rApp-X which owns policies of release policy group with option.1 in Table 7.2.2.1-1.

NOTE: An implementation of the Energy Saving policy type may apply energy saving only if there is no harm to traffic.

As for the A1 policy conflict use case #2 Policy objectives not uniquely identified as shown in clause 5.2, there would be some of potential solutions for operators to configure the A1 policy priority with option.3 in Table 7.2.2.1-1. In a case of UE-level policy vs. Slice SLA policy conflict scenario as shown in clause 5.2.2, both policies basically have different policy objective KPIs but some of them are identical KPIs in terms of UE level performance. In such case, operators would put higher priority for the UeLevelObjectives KPIs of “ulThroughput” or “dlThroughput” (as defined in A1TD [i.4], clause 6.3.3.4) than the SliceSlaObjectives KPIs of “maxDlThptPerUe” or “maxUlThptPerUe” (as defined in A1TD [i.4], clause 6.3.3.5) if operators think that UE-level policy should have higher priority than Slice SLA policy for entities in overlapping scope.

As for the A1 policy conflict use case #3 Policy resources preference not uniquely identified as shown in clause 5.3, there would be some of potential solutions for operators to configure the A1 policy priority with option.2 in Table 7.2.2.1-1. In a case of TSP policy with ueId specified vs. TSP policy with qosId specified for different “SHALL” preference cells as shown in clause 5.3.2, operators would put higher priority for the TspResource cellIdList (as defined in A1TD [i.4], clause 6.3.4.2) containing the specified cellId (cell-B) for the TSP policy with ueId specified than the TspResource cellList containing the specified cellId (cell-C) for the TSP policy with qosId specified if operators think that TSP policy with ueId specified should have higher priority than TSP policy with qosId specified.

7.2.2.2 Procedures

7.2.2.2.1 Conflict arbitration with priority

The figure below shows the message sequence of Option#2, A1 policy arbitrates with priority.

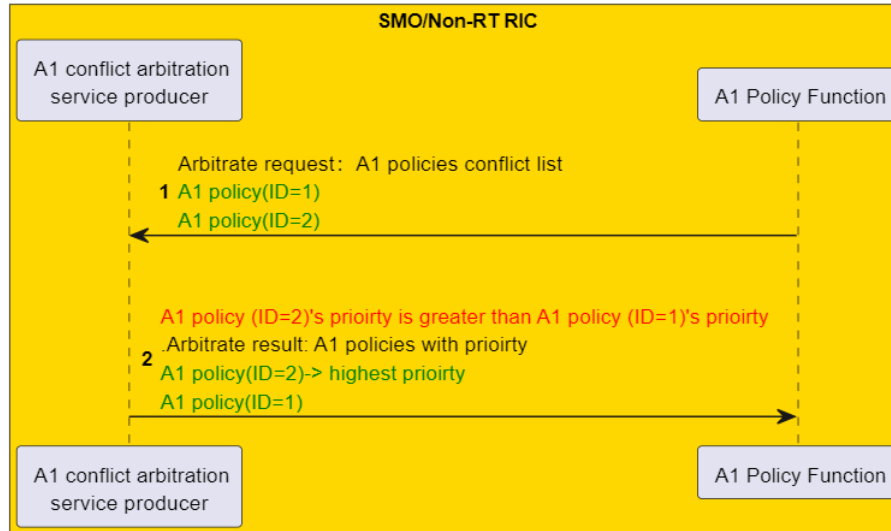


Figure 7.2.2.3.1- 1: Conflict arbitration with priority

7.2.2.2.2 Conflict arbitration with preemption protection flag

The figure below shows the message sequence of Option#2, A1 policy arbitrates with preemption protection flag.

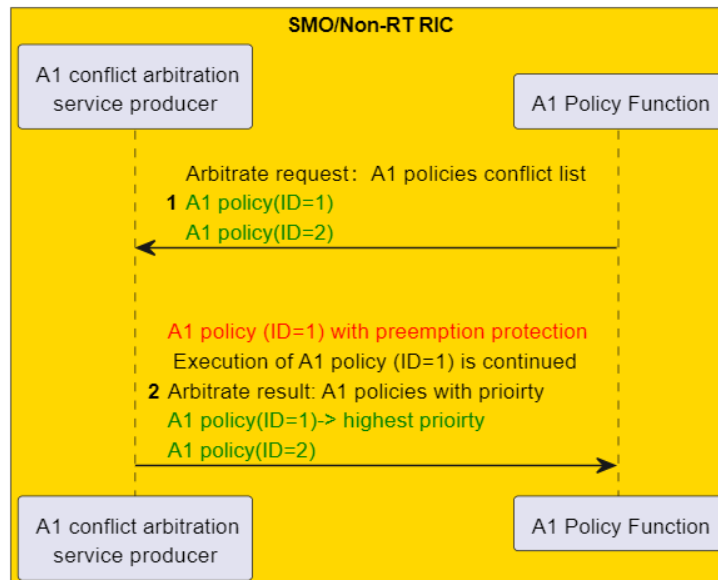


Figure 7.2.2.3.2- 1: Conflict arbitration with preemption protection flag

7.2.2.2.3 Conflict arbitration with preemption flag

The figure below shows the message sequence of Option#2, A1 policy arbitrates with preemption flag.

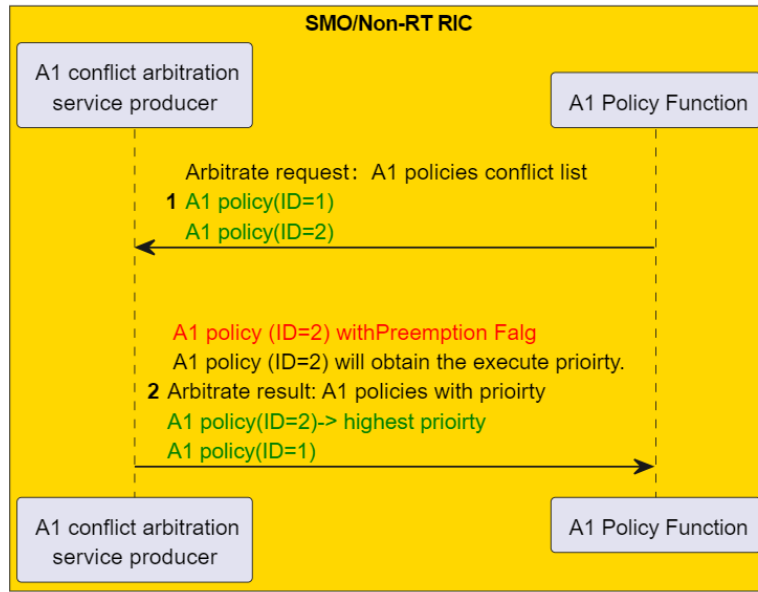


Figure 7.2.2.3.3- 1: Conflict arbitration with preemption flag

7.3 Procedures

7.3.1 General

There are different approaches for conflict avoidance.

Approach #1 is that the A1 policy function enforces A1 policies based on arbitration results by requesting rApps to delete, reject, or update an A1 policy. The A1 policy function implements conflict avoidance actions, such as rejecting a new create request from an rApp, request the rApp to delete an A1 policy or notifying rApps to update the A1 policy with proposed alternative A1 policies and conflict details information. In this approach, rApp serves as the lifecycle management (LCM) owner of A1 policies, while the A1 policy function can propose alternative A1 policies to avoid the conflicts.

Approach #2 is that send A1 policies with the information of A1 policy priorities and pre-emption information to Near-RT RIC.

Approach #3 is that SMO/Non-RT RIC provides an alternative set of conflict-free policies based on conflict detection information and conflict arbitration result.

An example of approaches for policy conflict avoidance in subset/superset overlapping scope is illustrated in Figure 7.3.1-1. In the referred figure, there are two policies (P1, P2) having subset/superset overlapping scope and conflicting situation. With case.1 as shown in the leftmost of the referred figure, approach1 is applied, exclusively one of the two policies would be selected to avoid policy conflict based on the arbitration result. In this approach, entities d and e in case.1-1 of the referred figure would not have applicable policy due to selection of the policy P1 with smaller policy scope while there are no such entities without applicable policy in case.1-2 of the referred figure with selection of the policy P2 with bigger policy scope. With case.2 as shown in the middle of the referred figure, approach2 is applied, multiple policies would be enforced simultaneously in Near-RT RIC with different priorities based on the arbitration result. In case.2-1 of the referred figure, the policy P1 has higher priority than the policy P2 and therefore the entities a, b and c in the overlapping scope are applied with higher policy P1 while in case.2-2 of the referred figure, the policy P2 has higher priority than the policy P1 and therefore all entities in the scope of the policy P2 are applied with higher policy P2. With case.3 as shown in the rightmost of the referred figure, approach1 or approach3 is applied, the policy P1

and/or the policy P2 would be modified into the policy P1' and/or the policy P2' to resolve conflicting situation by aligning with the same policy objectives to the ones of the highest prioritized policy. In case.3-1 of the referred figure, the policy objectives of the policy P2 would be changed into the policy P2' to be aligned with the ones of the policy P1 because the policy P1 has higher priority than the policy P2. In case.3-2 of the referred figure, the policy objectives of the policy P1 would be changed into the policy P1' to be aligned with the ones of the policy P2 because the policy P2 has higher priority than the policy P1.

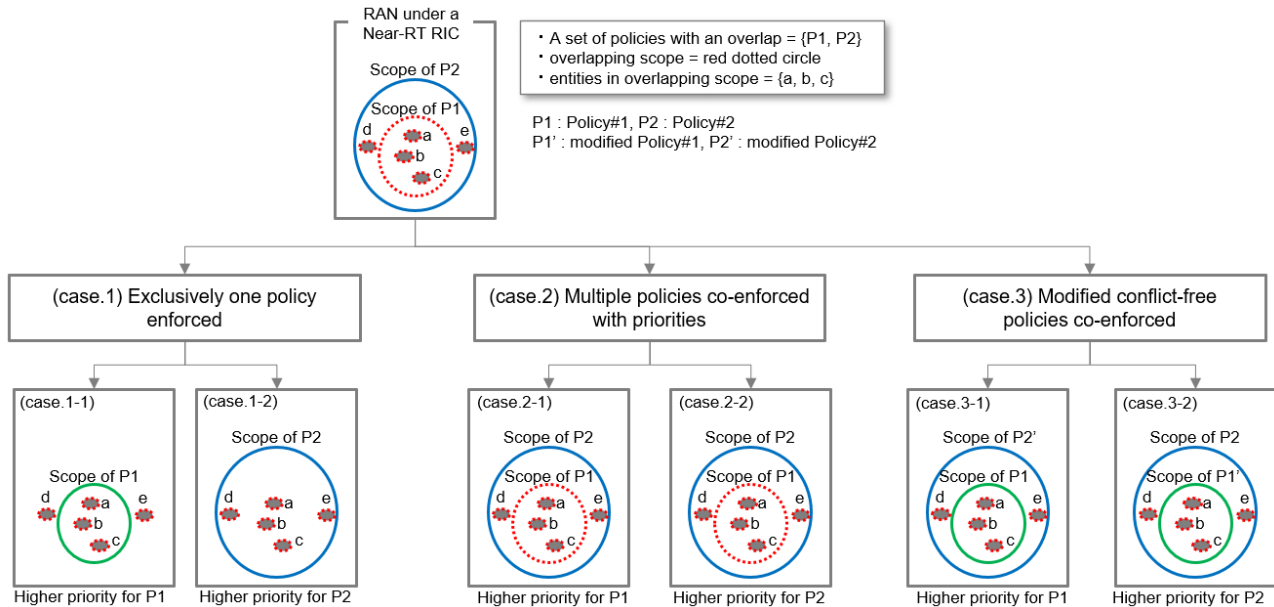


Figure 7.3.1- 1: An example of approaches for policy conflict avoidance in Subset/Superset overlapping scope.

An example of approaches for policy conflict avoidance in Jointset overlapping scope is illustrated in Figure 7.3.1-2. In the referred figure, there are two policies (P1, P2) having jointset overlapping scope and conflicting situation. With case.1 as shown in the leftmost of the referred figure, approach1 is applied, exclusively one of the two policies would be selected to avoid policy conflict based on the arbitration result. In this approach, entity e in case.1-1 of the referred figure would not have applicable policy due to selection of the policy P1 while entity d in case.1-2 of the referred figure would not have applicable policy due to selection of the policy P2. With case.2 as shown in the middle of the referred figure, approach2 is applied, multiple policies would be enforced simultaneously with different priorities based on the arbitration result. In case.2-1 of the referred figure, the policy P1 has higher priority than the policy P2 and therefore the entities a, b and c in the overlapping scope are applied with higher policy P1 while in case.2-2 of the referred figure, the policy P2 has higher priority than the policy P1 and therefore the entities a, b and c in the overlapping scope are applied with higher policy P2. With case.3 as shown in the rightmost of the referred figure, approach1 or approach3 is applied, the policy P1 and/or the policy P2 would be modified into the policy P1' and/or the policy P2' to resolve conflicting situation by aligning with the same policy objectives to the ones of the highest prioritized policy. In case.3-1 of the referred figure, the policy objectives of the policy P2 would be changed into the policy P2' to be aligned with the ones of the policy P1 because the policy P1 has higher priority than the policy P2. In case.3-2 of the referred figure, the policy objectives of the policy P1 would be changed into the policy P1' to be aligned with the ones of the policy P2 because the policy P2 has higher priority than the policy P1.

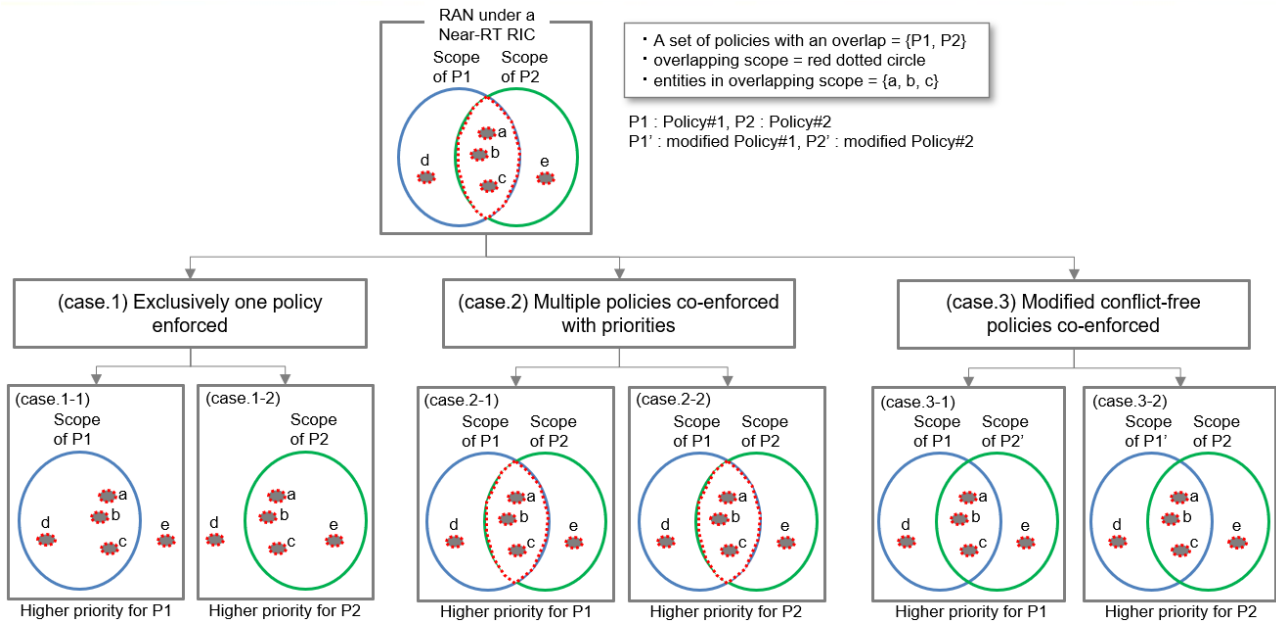


Figure 7.3.1- 2: An example of approaches for policy conflict avoidance in Jointset overlapping scope.

As for modifying the conflicting policies through arbitration result, it is noted that modifying the policy objectives would be a way to resolve the conflicting situation with overlapping scope while modifying the policy scopes and/or policy resources would be a way to remove the overlapping scope.

7.3.2 Approach #1. enforce A1 policy based on arbitration result

7.3.2.1 Reject the creation/update of an A1 policy

When an A1 policy create or update request with a lower priority among conflict A1 policies is received, SMO/Non-RT RIC could reject this A1 policy by providing a response with an A1 policy create or update failure.

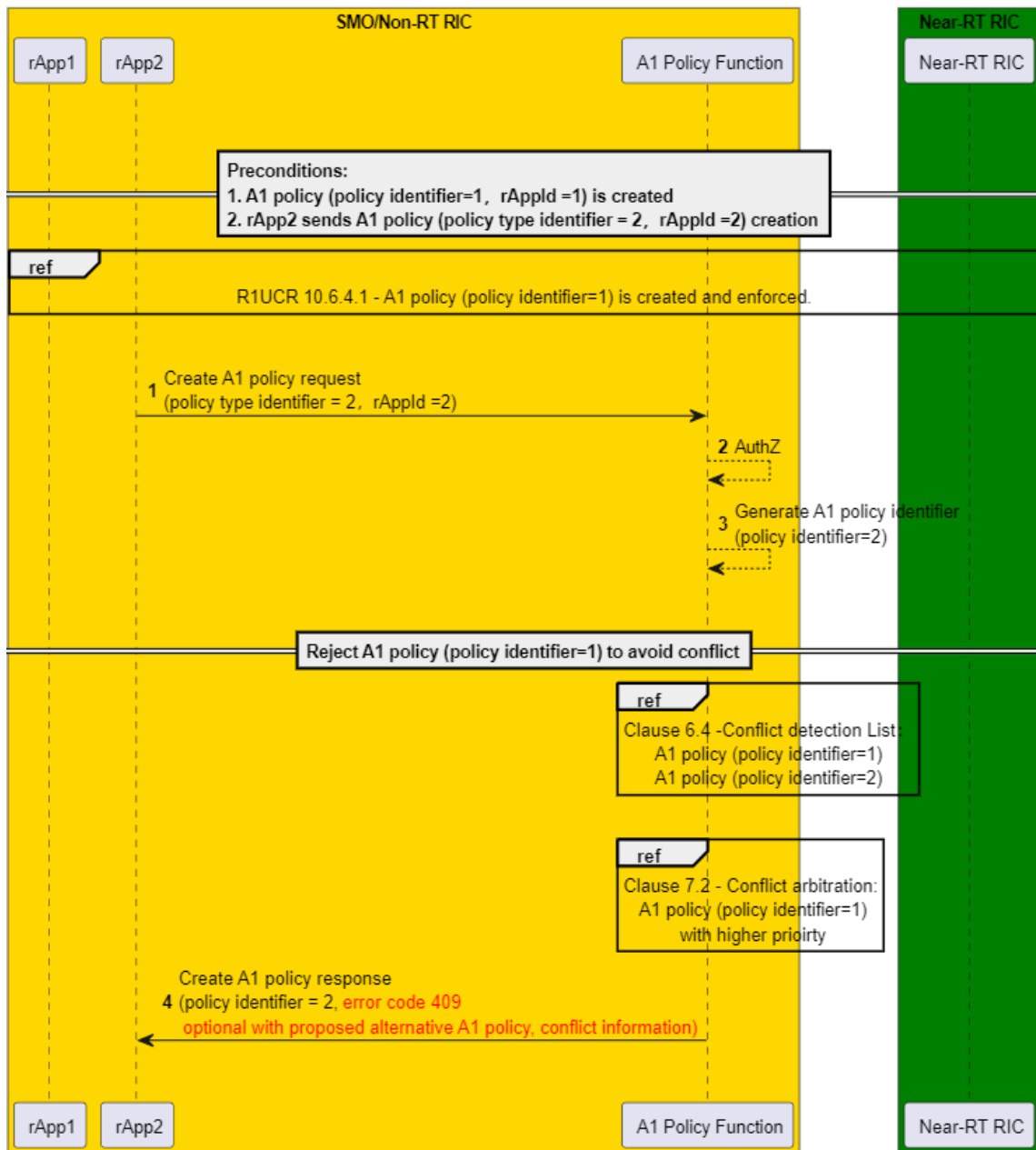


Figure 7.3.2.1- 1: Reject create/update of an A1 policy

7.3.2.2 Notify an rApp to retry

After a former A1 policy creation has been rejected due to conflict or an A1 policy previously causing conflict has been deleted or updated, and an URI for retry notification had been provided in the A1 policy creation message, the related rApp can be notified to retry the previously rejected or deleted A1 policy once the existing A1 policies that are causing the conflicts have been deleted or conflicts do not exist anymore. Once an rApp receives an A1 policy retry notification message, rApp can make the decision to resend an A1 policy as same as previous one, or send an updated A1 policy, or do not send an A1 policy at all. In extreme scenarios, there may be more than one previous rejected or deleted A1 policies, rApp can process all these A1 policies after receiving one A1 policy retry notification message.

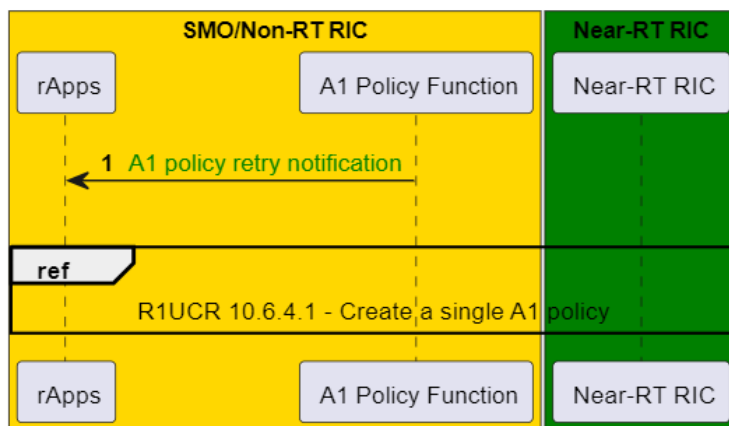


Figure 7.3.2.2- 1: Notify an rApp to retry

7.3.2.3 Terminate/update an A1 policy

When an existing A1 policy has a lower priority compared to an A1 policy requested for creation/updation, as determined by arbitration results, SMO/Non-RT RIC could request to delete/update this A1 policy by sending an A1 policy delete/update request notifications. Subsequently, the rApp should proceed with the delete/update of the corresponding A1 policy.

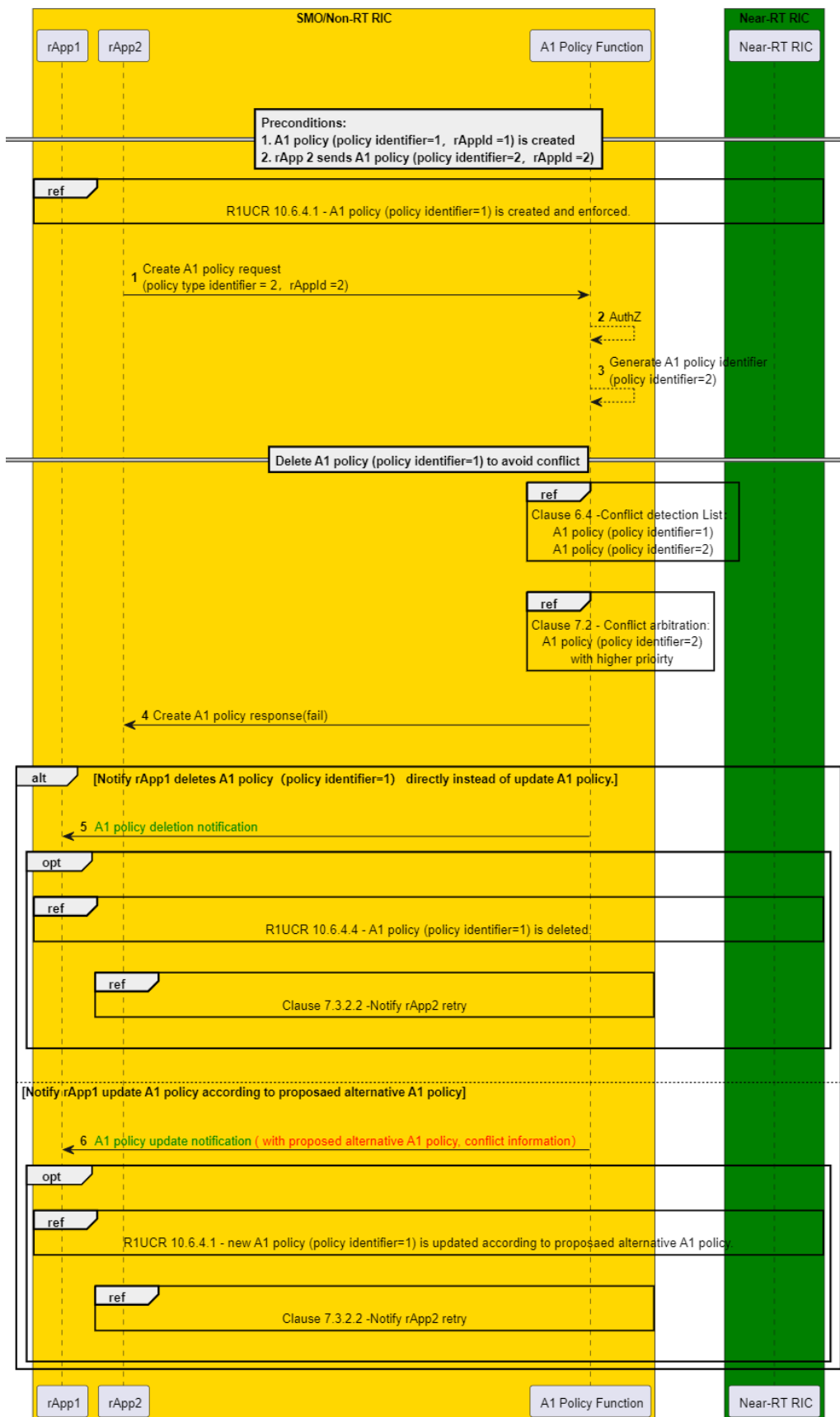


Figure 7.3.2.3- 1: Terminate/update an A1 policy

7.3.3 Approach #2. multiple policies co-enforced with priorities and pre-emption information

7.3.3.1 Near-RT RIC mitigate conflict with priorities and pre-emption information

Non-RT RIC can transmit A1 policy priority and preemption information to Near-RT RIC. In this case, rApp can query A1 policy priority and preemption information and provide this information in A1 policy create and update messages. Near-RT RICs employ this information for Near-RT RICs conflict mitigation. Near-RT RICs can reject policies or propose delete or update of A1 policies for conflict mitigation, and new A1 policy service operations needed to be introduced to support this.

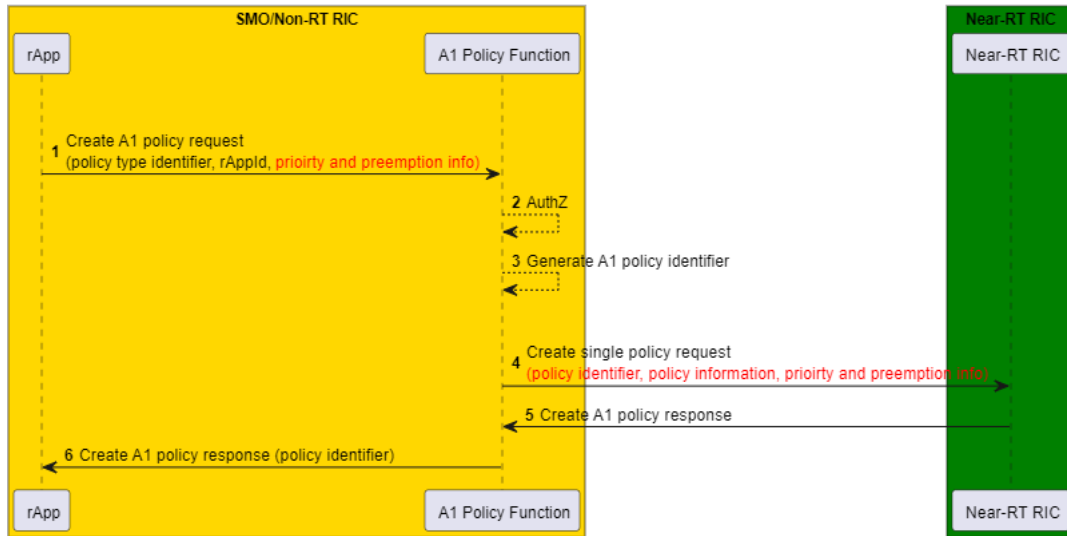


Figure 7.3.3.1- 1: Near-RT RIC Mitigate Conflict with Priorities

7.3.4 Approach #3. provide alternative conflict-free policies

7.3.4.1 Provide alternative conflict-free policies

In approach#3, SMO/Non-RT RIC framework can mitigate the conflict by modifying A1 policies base on arbitration result. A1 policy function is responsible for conflict avoidance base on conflict detection and conflict arbitration result. In this approach, A1 policy function is also responsible for maintaining the original A1 policies, the updated A1 policies, mapping relationship between original and updated A1 policies. A1 policy function can update the A1 policies to avoid conflict directly and notify rApps with updated A1 policies information.

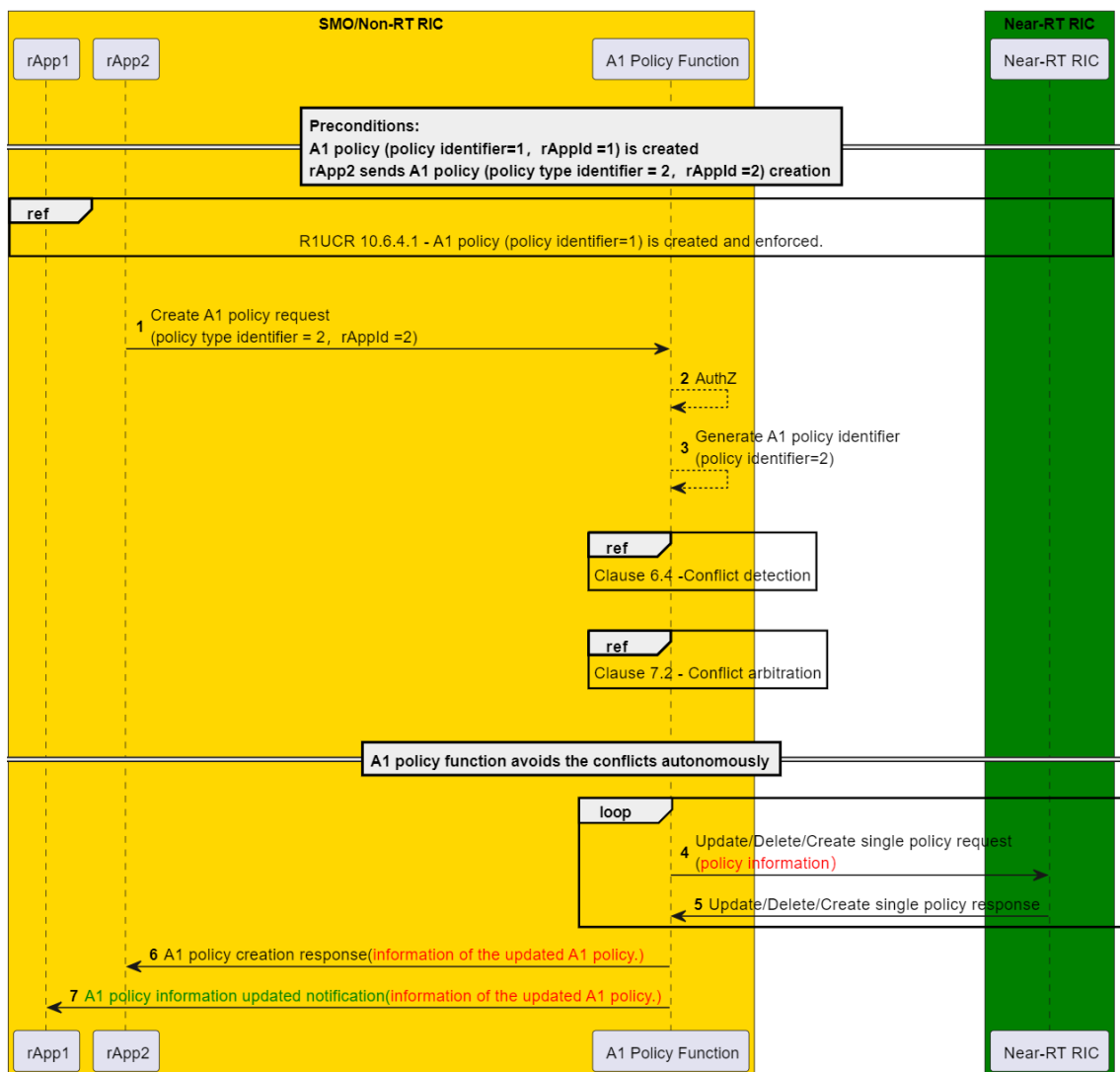


Figure 7.3.4.1- 1: Provide Alternative Conflict-free Policies

7.3.5 Analysis of approaches

Table 7.3.5- 1: Analysis of approaches

Approach	Prospect	Consideration
Approach #1 Enforce A1 Policy Base on Arbitration Result	- Provide conflict avoidance with enforcement preference in Non-RT RIC. - rApps are always responsible for the A1 policy creation, modification and termination which facilitates synchronization between rApps and the SMO/Non-RT RIC frameworks.	- The R1 interface needs to be adapted to meet the requirements. - rApp is always responsible for LCM of A1 policies.
Approach #2 Co-enforced policies with priorities	Provide with priorities and pre-emption information to Near-RT RIC to facilitate conflict mitigation.	The R1 and A1 interface needs to be adapted to meet the requirements.

		The usage of A1 policies priorities and pre-emption information (refer to clause 7.2.2.1) is not within the scope of Non-RT RIC.
Approach #3 Provide Alternative Conflict-free Policies	A solution may maximally meet the requirements of all A1 policies from different rApps, especially in autonomous network.	- The R1 interface needs to be adapted to meet the requirements. - The A1 policy function is responsible for maintaining the original A1 policies, updated A1 policies, and mapping relationship between the original and updated A1 policies. - SMO/Non-RT RIC framework will bypass the rApps to update the A1 policy, i.e., SMO/Non-RT RIC framework is responsible for LCM of A1 policies during conflict avoidance, rather than consistently relying on rApps. There is a possibility that the updated A1 policies may not be acceptable for Near-RT RICs and rApps.

8 Recommendations

8.1 Recommendations for A1 policy conflict related services

[REC-A1PCRS1]	A1 policy conflict detection service should support use cases of the A1 policy conflict detection service as shown in Table 6.4-1. (clause 6.4)
[REC-A1PCRS2]	A1 policy conflict detection service should provide overlap information (clause 6.2.4) and information of conflicting situation (clause 6.3.3) for detected sets of potential conflicting policies. (clause 6.4)
[REC-A1PCRS3]	A1 policy conflict detection service should provide the capabilities to uniquely identify the detected conflicts in the SMO/Non-RT RIC. (clause 6.3.1)

[REC-A1PCRS4]	A1 policy conflict detection service producer should have capability to collect information about existing A1 policies with use of the A1 policy management services. (clause 6.4)
[REC-A1PCRS5]	A1 policy conflict arbitration service should support the configuration of Influencing factors of A1 policy priorities (clause 7.2.2.1.1)
[REC-A1PCRS6]	A1 policy conflict arbitration service should provide information about priorities for a set of conflicting policies. (clause 7.2)
[REC-A1PCRS7]	A1 policy conflict arbitration service should provide the A1 policy priority and preempt information of A1 policies. (clause 7.2.2.1.2)

8.2 Recommendations for A1 policy management services

[REC-R1-A1P-FUN1]	The A1 policy management service should reject a create A1 policy request or an update A1 policy request if the requested policy has a conflict with any of the existing policies. Reject response should provide a relevant error code and error reason (refer to clause 7.3.2.1), if approach #1 is applied.
[REC-R1-A1P-FUN2]	A1 policy management service(s) should support the capability to provide the conflicting information in the reject response if a create A1 policy request or an update A1 policy request is rejected due to conflicts (refer to clause 7.3.2.1), if approach #1 is applied.
[REC-R1-A1P-FUN3]	A1 policy management service(s) should support the capability to propose alternative A1 policy if a create A1 policy request or an update A1 policy request is rejected due to conflicts (refer to clause 7.3.2.1), if approach #1 is applied.
[REC-R1-A1P-FUN4]	The A1 policy management service should support the capability to notify the registered consumer when a policy is deleted or updated (refer to clause 7.3.2.2), if approach #1 is applied.

[REC-R1-A1P-FUN5]	The A1 policy management service should support the capability to notify the owner of an A1 policy about the proposal to delete or update an A1 policy optionally along with reason behind the proposal, if approach #1 is applied. Note: Reason for such proposal could be conflict avoidance when an existing A1 policy has a lower priority compared to a requested A1 policy for creation or updating, as determined by arbitration results (refer to clause 7.3.2.3).
[REC-R1-A1P-FUN6]	The A1 policy management service (s) should support to provide A1 policy priority and preemption information in an A1 policy creation or update message. (refer to clause 7.3.3.1), if approach #2 is applied.
[REC-R1-A1P-FUN7]	A1 policy management service should support the capability to notify the owner of an A1 policy about the forced deletion/updating of an A1 policy along with relevant reason. In case of update, notification should provide the updated A1 policy, if approach #3 is applied. Note: Reason for such forced deletion/updating of an A1 policy could be conflict mitigation when an existing A1 policy has a lower priority compared to a requested A1 policy for creation or updating, as determined by arbitration results (refer to clause 7.3.4.1).

8.3 Note for recommendations in terms of formulation for A1 policy conflict related services

Through the A1 policy conflict mitigation SI, we propose to support two A1 policy conflict related services in the TR below:

- A1 policy conflict detection service
- A1 policy conflict arbitration service

These are surely the A1-related services but not obvious if they should be regarded as part of the A1 policy management service(s) as defined in Non-RT-RIC-ARCH [i.2], clause 7.4.1 because the A1 policy conflict related services have totally new and possibly independent functions from such fundamental ones as the A1 policy management service. There would be options to formulate the A1 policy conflict related services:

- Option.1 A1 policy conflict related services as part of A1 policy management service(s)
- Option.2 Individual A1 policy conflict related services separate from A1 policy management service(s)
- Option.3 Set of A1 policy conflict related services separate from A1 policy management service(s)

The formulation of the A1 policy conflict related services would be an item for further investigation to align with existing services and other related items like the R1 Services-Related Conflict Mitigation WI.

Change history / Change request (history)

Date	Revision	Description
2025 March	01.00	Published as Final version 01.00